Applied Machine Learning
Unit 05 — Lecture 11 Notes
Multiple Linear Regression

Tofik Ali

Autumn 2026

Read this first

These notes are written so that you can learn the material *without* having been in the lecture. Every concept is developed from scratch, every number you see was produced by running the code shown beside it, and every figure was generated from real computation rather than drawn by hand.

You already know simple linear regression: one predictor, a line, the two normal equations, least squares, R^2. This lecture generalises all of it to p predictors at once. The algebra becomes matrix algebra, and one line of matrix calculus replaces two pages of summation. Nothing you learned is discarded.

What is genuinely new is not the algebra. It is that a coefficient in a multiple regression is a **conditional** statement — "the effect of this predictor holding the others fixed" — and that this makes it behave in ways that will surprise you.

The single result to carry out of the lecture: on 442 diabetes patients, total serum cholesterol has a regression slope of $+\mathbf{0.4723}$ on its own and $-\mathbf{1.0900}$ once the other nine measurements are in the model. Not smaller. The opposite sign, and 2.3 times the magnitude. Nothing was done to the column. Work through Section 5.2 with a Python session open until you can say exactly why, and can predict when it will happen again.

Contents

1 How to use these notes **2**
1.1 What you need . . . 3
1.2 Learning outcomes . . . 4

2 The model and the design matrix **4**
2.1 From one predictor to p . . . 4
2.2 The example we will work completely . . . 5

3 Deriving the normal equations **6**
3.1 The objective . . . 6
3.2 The matrix calculus behind the gradient . . . 6
3.3 The same gradient, from first principles . . . 8
3.4 Setting the gradient to zero . . . 9
3.5 It contains the simple case . . . 9
3.6 When is $X^\top X$ invertible? . . . 9
3.7 Failure 1, measured . . . 10
3.8 Failure 2, measured . . . 10
3.9 The geometry: residuals are orthogonal to the predictors . . . 11

4 The five flats, worked completely **11**
4.1 Step 1: form $X^\top X$ and $X^\top y$ . . . 11
4.2 Step 2: invert $X^\top X$ . . . 12
4.3 Step 3: multiply out . . . 12
4.4 Step 4: check the fit . . . 12
4.5 The same thing in code . . . 13
4.6 A shortcut for hand calculation: centre first . . . 13

5 Interpreting the coefficients **14**
5.1 "Holding the others fixed" . . . 14
5.2 The sign flip, on real data . . . 15
5.3 Where the difference goes: the omitted-variable formula . . . 16
5.4 Partial regression coefficients and added-variable plots . . . 17

6 Standardized (beta) coefficients **18**
6.1 Why raw coefficients cannot be compared . . . 18
6.2 Measured, and the ranking changes . . . 18

7 Multicollinearity **19**
7.1 Near-dependence, and what it costs . . . 19
7.2 Coefficient instability, measured by the bootstrap . . . 20
7.3 Predicting is fine. Explaining is not. . . . 21
7.4 What to do about it . . . 23

8 Missing data in a regression **23**
8.1 Why it is worse here than you expect . . . 23
8.2 Five strategies, measured . . . 24
8.3 Adding a missing-indicator column . . . 25
8.4 An imputer is a learned step . . . 25

9 Model validation **26**
9.1 Three questions, three checks . . . 26
9.2 Residual diagnostics . . . 26
9.3 R^2 always rises; adjusted R^2 does not . . . 27
9.4 Everything in one pipeline . . . 28

10 Summary **29**

11 Practice questions **29**

12 Solutions **31**

13 References and further reading **34**

1 How to use these notes

Read in order. Section 4 is the arithmetic backbone of everything after it, and Sections 5 to 7 are one continuous argument: what a coefficient means, why two coefficients cannot be compared, and when a coefficient should not be reported at all. The one new piece of mathematics is the derivative of a quadratic form, done carefully in Section 3.

Everything here assumes what the earlier regression lecture assumed: least squares with one predictor, the residual sum of squares, and R^2. It also uses two ideas from the preprocessing

unit — standardisation, and the rule that anything learned from data must be learned inside the cross-validation fold.

There are four kinds of box. **Key idea** — the one sentence to remember a month from now. **Worked example** — a calculation done by hand, with small numbers, before any library is used. **Caution** — the specific mistake that section exists to prevent. **Try it yourself** — something to run, with the expected output given so you can check.

Code appears in a framed block, and directly beneath it a shaded block showing what that code *actually printed*. If you run it and get something different, trust your machine and tell me.

1.1 What you need

```
pip install numpy scipy scikit-learn matplotlib
```

Every number in these notes was produced with:

```
import numpy, scipy, sklearn, matplotlib
print("numpy", numpy.__version__, "| scipy", scipy.__version__,
      "| sklearn", sklearn.__version__,
      "| matplotlib", matplotlib.__version__)
```

```
numpy 2.2.6 | scipy 1.15.3 | sklearn 1.7.2 | matplotlib 3.10.9
```

The running example is scikit-learn's bundled diabetes table, loaded in its **original units**. This matters: the default `load_diabetes()` returns columns that have already been centred and scaled to a common size, which is convenient for optimisation demonstrations and useless for reading coefficients.

```
from sklearn.datasets import load_diabetes, load_wine
dia = load_diabetes(scaled=False) # ORIGINAL UNITS
X, y = dia.data, dia.target
```

```
load_diabetes(scaled=False)  442 rows x 10 features, continuous target
load_wine                    178 rows x 13 features
missing cells anywhere: 0

feature   meaning                          mean       sd
age       age in years                   48.518   13.109
sex       sex, coded 1 / 2                1.468    0.500
bmi       body mass index                26.376    4.418
bp        mean arterial pressure         94.647   13.831
s1        total serum cholesterol       189.140   34.608
s2        low-density lipoproteins      115.439   30.413
s3        high-density lipoproteins      49.788   12.934
s4        total cholesterol / HDL         4.070    1.290
s5        log serum triglycerides         4.641    0.522
s6        blood sugar level              91.260   11.496

the pre-scaled load_diabetes() has every column at sd 0.0476
   -- useful for optimisation demos, useless for reading coefficients
```

Two structural facts about that table will matter a great deal later. `s4` is *by definition* `s1` divided by `s3`, and `s2` is a component of `s1`. Five of the ten columns are five views of one blood sample.

Do not reach for `fetch_*`

Every `fetch_*` loader downloads over HTTPS, and on the campus network those downloads fail with a certificate error you cannot fix from your machine. Nothing in this course depends on one: use the bundled loaders (`load_diabetes`, `load_wine`, `load_breast_cancer`, `load_iris`, `load_digits`) or a fixed-seed synthetic set.

One seed, and one sweep — the convention for this lecture

The data is `load_diabetes(scaled=False)` above, plus `load_wine` and the five-flat table written out by hand. None of it is randomly generated, so there is no dataset seed to worry about.

Everything arbitrary — the eight-patient subset, every train/test split, every simulated pattern of missing cells, the imputer, the junk columns, the designed collinear pair — uses **seed** 0, and every listing shows it.

One thing in this lecture is deliberately *not* collapsed to a single seed. Section 7.2 refits the model on 2000 bootstrap resamples, and the *spread across those* 2000 *fits is the entire result*: it is how we will show that a coefficient under collinearity is not a number you can honestly report. That is a **sweep**, not an arbitrary choice, and collapsing it would delete the demonstration. It is written as `range(0, 2000)` — derived from the same seed 0 — so the sweep stays a sweep and still reproduces exactly.

Learn the distinction, because it recurs: a seed you picked arbitrarily should be fixed and stated; a set of seeds you are deliberately varying is an experiment, and its variability is data.

1.2 Learning outcomes

By the end of this lecture you should be able to:

1. Write a multiple linear regression in matrix form, and derive $\hat{\beta} = (X^\top X)^{-1} X^\top y$ by differentiating the residual sum of squares, stating the condition under which the inverse exists and naming the two ways it fails (CO2).
2. Compute $X^\top X$, its inverse and $\hat{\beta}$ by hand for a small table with two predictors, verify the result against the orthogonality condition $X^\top e = 0$, and reproduce it with `LinearRegression` (CO2).
3. Interpret a multiple-regression coefficient as a conditional quantity, explain and predict the change of sign that occurs when a correlated predictor enters the model, and construct the corresponding partial regression coefficient and added-variable plot (CO2, CO3).
4. Compute standardized (beta) coefficients and say what they can and cannot be used for (CO3).
5. Diagnose multicollinearity using variance inflation factors and bootstrap resampling, and distinguish the situations in which it destroys an explanation from those in which it leaves a prediction intact (CO3).
6. Handle missing data in a regression setting: quantify the cost of complete-case analysis, choose and justify an imputation strategy, and place it correctly inside a validated pipeline (CO2).
7. Validate a fitted model using residual diagnostics, held-out performance and adjusted R^2, and explain why R^2 alone cannot be used to choose between models of different size (CO3).

2 The model and the design matrix

2.1 From one predictor to p

The multiple linear regression model for n observations and p predictors is

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \varepsilon_i, \qquad i = 1, \ldots, n. \tag{1}$$

Stack the n equations and it becomes a single matrix equation:

$$y = X\beta + \varepsilon, \qquad X = \begin{pmatrix} 1 & x_{11} & \cdots & x_{1p} \\ 1 & x_{21} & \cdots & x_{2p} \\ \vdots & \vdots & & \vdots \\ 1 & x_{n1} & \cdots & x_{np} \end{pmatrix}, \tag{2}$$

where y is $n \times 1$, X is $n \times (p+1)$, β is $(p+1) \times 1$ and ε is $n \times 1$. The matrix X is called the **design matrix**. The leading column of ones is what makes β_0 an ordinary coefficient rather than a special case: with it in place, every formula below treats the intercept exactly like any other parameter.

Key idea

"Linear" means linear *in the parameters*, not in the predictors. $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 \log x$ is a multiple linear regression — put x^2 and $\log x$ in as columns. $y = \beta_0 x^{\beta_1}$ is not, because β_1 appears in an exponent.

Geometrically, one predictor fits a line in the plane, two fit a plane in three dimensions, and p fit a p-dimensional hyperplane in $(p+1)$ dimensions. The fitting criterion is unchanged: choose the hyperplane that minimises the sum of squared vertical distances to the data.

2.2 The example we will work completely

Five flats, two predictors

Prices in lakh of rupees. x_1 is the number of bedrooms; x_2 is the floor area, in units of $50\,\mathrm{m}^2$ (so $x_2 = 3$ means $150\,\mathrm{m}^2$).

flat	A	B	C	D	E
bedrooms x_1	1	2	3	4	5
area x_2	2	3	4	3	4
price y	8	13	15	11	12

Before computing anything, look at the pairs that share a floor area. B and D are both $150\,\mathrm{m}^2$: B has two bedrooms and costs 13, D has four and costs 11, so within that area an extra bedroom is worth -1 lakh. C and E are both $200\,\mathrm{m}^2$: three bedrooms at 15, five at 12, so -1.5 per bedroom.

Now ignore the area column entirely and regress price on bedrooms alone. With $\bar{x}_1 = 3$ and $\bar{y} = 11.8$, $S_{11} = 10$ and $S_{1y} = 6$, so the slope is $+0.6$: **more bedrooms, higher price**.

Both statements are true of the same five flats. The rest of these notes is about how to hold them in your head at the same time.

The design matrix and target for this example are

$$X = \begin{pmatrix} 1 & 1 & 2 \\ 1 & 2 & 3 \\ 1 & 3 & 4 \\ 1 & 4 & 3 \\ 1 & 5 & 4 \end{pmatrix}, \qquad y = \begin{pmatrix} 8 \\ 13 \\ 15 \\ 11 \\ 12 \end{pmatrix}, \qquad n = 5, \quad p = 2.$$

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.;Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.;Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.;Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.

3 Deriving the normal equations

3.1 The objective

Least squares chooses the coefficient vector b that minimises the residual sum of squares. In matrix notation,

$$S(b) = \sum_{i=1}^{n} (y_i - x_i^\top b)^2 = (y - Xb)^\top (y - Xb). \tag{3}$$

Expand the product. Use $(Xb)^\top = b^\top X^\top$ on the left factor,

$$S(b) = (y^\top - b^\top X^\top)\,(y - Xb),$$

and then multiply the two brackets out term by term, keeping every factor in the order in which it appears — matrix multiplication does not commute, so nothing may be moved past anything else:

$$S(b) = \underbrace{y^\top y}_{(1\times n)(n\times 1)} - \underbrace{y^\top X b}_{(1\times n)(n\times q)(q\times 1)} - \underbrace{b^\top X^\top y}_{(1\times q)(q\times n)(n\times 1)} + \underbrace{b^\top X^\top X b}_{(1\times q)(q\times q)(q\times 1)},$$

where $q = p + 1$. Every one of the four terms is 1×1, as it must be: $S(b)$ is a single number. Checking the shapes like this is the cheapest way to catch an algebra slip, and it costs a few seconds.

The two middle terms are not merely similar — they are the same number. Each is 1×1, and a 1×1 matrix is its own transpose, so applying $(UVW)^\top = W^\top V^\top U^\top$,

$$y^\top X b \;=\; (y^\top X b)^\top \;=\; b^\top X^\top (y^\top)^\top \;=\; b^\top X^\top y.$$

Therefore

$$S(b) = y^\top y \;-\; 2\, b^\top X^\top y \;+\; b^\top X^\top X\, b. \tag{4}$$

Three terms: a constant, a term *linear* in b, and a **quadratic form** in b with the symmetric matrix $X^\top X$.

Caution

The step $y^\top X b = b^\top X^\top y$ looks like two matrices being swapped past each other, and it is not. It is a *transpose*, and it is legal only because the object is 1×1. The same move applied to a general product is simply false: $AB \neq BA$, and $AB \neq B^\top A^\top$ either. Whenever you are tempted to reorder a matrix product, check the shape first. If it is not 1×1, you may not.

3.2 The matrix calculus behind the gradient

We need the gradient of (4) with respect to the vector b. Before the two rules, it is worth being precise about what the notation means, because almost every sign and transpose error in this subject comes from leaving it vague.

What $\partial/\partial b$ means here. S is a *scalar* and b is a q-vector, so $\partial S/\partial b$ is the q-vector whose kth entry is the ordinary partial derivative $\partial S/\partial b_k$. That is all. Any identity below can be proved by writing out that one entry, and when an identity looks doubtful, writing out that one entry is how you settle it.

Why the objective is a scalar, and why that matters. You may reasonably ask for the derivative of a matrix product such as AB with respect to b. The answer is a three-index object: if AB is $m \times n$ and b has q entries, the derivative has $m \times n \times q$ components and is not a matrix at all. This is why, in this subject, we only ever differentiate a *scalar* with respect to a vector — and it is why the loss is always built to be a single number. The one intermediate case that does stay matrix-shaped is a vector differentiated by a vector: with X constant, $\partial(Xb)/\partial b = X$, a Jacobian whose row i is the derivative of output i. (That convention, output down the rows, is called *numerator layout*. Adopt it and stay in it; switching layout halfway through a derivation is what produces a stray transpose at the end.)

The product rule, when both factors depend on b. For completeness, if A and B both vary with b_k then

$$\frac{\partial(AB)}{\partial b_k} = \frac{\partial A}{\partial b_k} B \;+\; A \frac{\partial B}{\partial b_k},$$

with the **order of every factor preserved**. We will not need this in the present derivation — here X and y are constant data and only b varies — but it is the rule behind backpropagation, and the order is the part that matters.

Now the two rules this derivation actually uses.

Linear term. For a constant vector c, $b^\top c = \sum_j b_j c_j$, so $\partial(b^\top c)/\partial b_k = c_k$, that is

$$\frac{\partial}{\partial b}(b^\top c) = c. \tag{5}$$

Quadratic form, general A. Write $b^\top A b = \sum_j \sum_k b_j A_{jk} b_k$. The index m appears in two places — once as j and once as k — so differentiating with respect to b_m picks up one contribution from each:

$$\frac{\partial}{\partial b_m}(b^\top A b) = \sum_k A_{mk} b_k \;+\; \sum_j b_j A_{jm} = (Ab)_m + (A^\top b)_m,$$

that is

$$\frac{\partial}{\partial b}(b^\top A b) = (A + A^\top) b \qquad \text{for } \textit{any} \text{ square } A. \tag{6}$$

Quadratic form, symmetric A. If and only if $A = A^\top$ do the two contributions coincide, and (6) collapses to the form you will see quoted everywhere:

$$\frac{\partial}{\partial b}(b^\top A b) = 2Ab \qquad (A \text{ symmetric}). \tag{7}$$

We are entitled to (7) here because $X^\top X$ is symmetric for *any* X whatever: $(X^\top X)^\top = X^\top (X^\top)^\top = X^\top X$. That is a fact we have earned, not an assumption.

Caution

$\partial(b^\top A b)/\partial b = 2Ab$ is **wrong** unless A is symmetric. Take $A = \left(\begin{smallmatrix} 1 & 2 \\ 0 & 1 \end{smallmatrix}\right)$ and $b = (3, 5)^\top$. Then $b^\top A b = b_1^2 + 2b_1 b_2 + b_2^2$, whose gradient is $(2b_1 + 2b_2,\; 2b_1 + 2b_2)^\top = (16, 16)^\top$ — which is $(A + A^\top)b$, and *not* $2Ab = (26, 10)^\top$. The scalar analogy $\mathrm{d}(ax^2)/\mathrm{d}x = 2ax$ is what misleads people here. Check symmetry before you use the factor of two.

Applying (5) with $c = X^\top y$ and (7) with $A = X^\top X$ to the three terms of (4) — the first is constant and contributes nothing — gives

$$\nabla S(b) = -2\, X^\top y + 2\, X^\top X\, b. \tag{8}$$

3.3 The same gradient, from first principles

The rules above are convenient, not necessary. It is worth doing the whole differentiation once in plain summation notation, with no matrix calculus at all, both as a check and because the result arrives in a form that hands us the next section for free.

Write the ith residual as $e_i = y_i - \sum_j X_{ij} b_j$, so that $S(b) = \sum_i e_i^2$. Differentiate with respect to a single coefficient b_k using the chain rule, noting that $\partial e_i / \partial b_k = -X_{ik}$:

$$\frac{\partial S}{\partial b_k} = \sum_{i=1}^{n} 2 e_i \frac{\partial e_i}{\partial b_k} = -2 \sum_{i=1}^{n} X_{ik}\, e_i = -2 \left(X^\top e\right)_k .$$

Collecting the q components into a vector,

$$\nabla S(b) = -2\, X^\top e = -2\, X^\top (y - Xb) = -2\, X^\top y + 2\, X^\top X b, \tag{9}$$

which is (8) again. Two things are worth noticing. First, no rule was quoted: this is the ordinary chain rule applied q times. Second, the middle form $\nabla S = -2X^\top e$ says that **the gradient is (minus twice) the correlation of the residual with each predictor column** — so setting it to zero is literally the statement $X^\top e = 0$, which is the geometry of Section 3.9 arriving a page early.

Try it yourself

Verify the gradient numerically before you trust it. Central differences approximate $\partial S / \partial b_k$ by $[S(b + he_k) - S(b - he_k)]/2h$, and should agree with the formula to several decimals at *any* b, not only at the optimum.

```
import numpy as np
x1 = np.array([1, 2, 3, 4, 5]); x2 = np.array([2, 3, 4, 3, 4])
y = np.array([8, 13, 15, 11, 12])
X = np.c_[np.ones(5), x1, x2]

def S(b): return float((y - X @ b) @ (y - X @ b))
def grad(b): return -2 * X.T @ y + 2 * X.T @ X @ b

def num_grad(b, h=1e-6):
    return np.array([(S(b + h*e) - S(b - h*e)) / (2*h) for e in np.eye(3)])

b = np.array([1.0, 1.0, 1.0]) # an arbitrary, non-optimal b
print("S(b) =", S(b))
print("formula =", grad(b))
print("finite diff =", np.round(num_grad(b), 6))

bhat = np.linalg.inv(X.T @ X) @ X.T @ y
print("beta-hat =", bhat)
print("grad(bhat) =", np.round(grad(bhat), 12))
print("S(bhat) =", S(bhat))
```

```
S(b)        = 127.0
formula     = [ -46. -122. -148.]
finite diff = [ -46. -122. -148.]
beta-hat    = [ 2. -1.  4.]
grad(bhat)  = [0.e+00 1.e-12 1.e-12]
S(bhat)     = 4.0
```

At $b = (1, 1, 1)$ the gradient is large and negative in every component: the objective falls steeply in all three directions, and $S = 127$. At $\hat\beta = (2, -1, 4)$ it is zero to twelve decimals and $S = 4$, which is the SSE you will reproduce by hand in Section 4. Make this check a

habit — a finite-difference comparison catches a dropped factor of two or a missing transpose in about a minute, and it works on objectives far too awkward to differentiate on paper.

3.4 Setting the gradient to zero

At a stationary point $\hat{\beta}$ we need $\nabla S(\hat{\beta}) = 0$:

$$\boxed{X^\top X\,\hat{\beta} = X^\top y} \tag{10}$$

These are the **normal equations**: $p+1$ linear equations in $p+1$ unknowns. If $X^\top X$ is invertible they have the unique solution

$$\boxed{\hat{\beta} = (X^\top X)^{-1} X^\top y} \tag{11}$$

It is a minimum and not a saddle or a maximum because the Hessian is

$$\nabla^2 S(b) = 2\,X^\top X,$$

and for any vector v, $v^\top (X^\top X) v = \|Xv\|^2 \geq 0$, so $X^\top X$ is positive semi-definite. S is therefore convex, and every stationary point is a *global* minimum. This is why linear regression has no local minima and no learning rate: unlike a neural network, it is solved, not searched.

Key idea

$\hat{\beta} = (X^\top X)^{-1} X^\top y$ is one line of matrix calculus: differentiate $y^\top y - 2b^\top X^\top y + b^\top X^\top X b$, get $-2X^\top y + 2X^\top X b$, set it to zero. Everything else in this section is checking that the inverse exists.

3.5 It contains the simple case

Put $p = 1$. Then

$$X^\top X = \begin{pmatrix} n & \sum x \\ \sum x & \sum x^2 \end{pmatrix}, \qquad X^\top y = \begin{pmatrix} \sum y \\ \sum xy \end{pmatrix},$$

and the two rows of (10) are $nb_0 + b_1 \sum x = \sum y$ and $b_0 \sum x + b_1 \sum x^2 = \sum xy$ — exactly the two normal equations you derived for the one-predictor case, and solving them gives $b_1 = S_{xy}/S_{xx}$ and $b_0 = \bar{y} - b_1 \bar{x}$. **Nothing has been replaced. The matrix form is the same equations, written once.**

3.6 When is $X^\top X$ invertible?

Key idea

$X^\top X$ is invertible if and only if X has **full column rank**, that is $\operatorname{rank}(X) = p + 1$.

Proof. Suppose $X^\top X v = 0$ for some vector v. Multiply on the left by $v^\top$: $v^\top X^\top X v = \|Xv\|^2 = 0$, and a norm is zero only when the vector is, so $Xv = 0$. Conversely, $Xv = 0$ immediately gives $X^\top X v = 0$. Hence

$$\operatorname{null}(X^\top X) = \operatorname{null}(X), \qquad \text{and so} \qquad \operatorname{rank}(X^\top X) = \operatorname{rank}(X).$$

$X^\top X$ is a $(p+1) \times (p+1)$ matrix, so it is invertible exactly when its rank is $p+1$, which by the display is exactly when X has $p+1$ linearly independent columns. □

There are exactly two ways to lose full column rank.

Failure 1: perfect collinearity. Some column of X is an exact linear combination of the others. This is a property of the data, and it happens more often than you would expect: a duplicated column; a total alongside all of its parts; the same quantity recorded in two units; all k indicator columns of a categorical variable together with the intercept (the dummy variable trap).

Failure 2: $p + 1 > n$. More parameters than observations. Then $\operatorname{rank}(X) \le \min(n, p+1) = n < p + 1$ *whatever the numbers in the table are.* This is the regime of gene expression data, text features, and any table that is wider than it is tall.

3.7 Failure 1, measured

Add a third column $x_3 = x_1 + x_2$ to the five flats. Nothing is wrong with the data — the third column is a perfectly sensible quantity — but the matrix is now rank deficient.

```
x3 = x1 + x2
Xd = np.c_[np.ones(5), x1, x2, x3]
print(np.linalg.matrix_rank(Xd), np.linalg.det(Xd.T @ Xd))
np.linalg.inv(Xd.T @ Xd)
```

```
added a third column x3 = x1 + x2
shape (5, 4)  rank 3  (needs 4)
det(X'X) = 0.000000e+00
np.linalg.inv  ->  LinAlgError: Singular matrix
LinearRegression still answers: [ 2. -2.  3.  1.]
a DIFFERENT coefficient vector: [ 2. -1.  4.  0.]
max |difference in predictions| 0.00e+00
condition number of X'X: 7.692e+18
```

Read the last four lines carefully. `np.linalg.inv` raised, as it should. `LinearRegression` did *not* raise: it calls `scipy.linalg.lstsq`, which returns the minimum-norm solution of an under-determined system instead of failing. And the coefficient vector it returned is not the only one — adding c to b_1 and b_2 and subtracting c from b_3 changes no prediction by 10^{-16}, because $x_1 + x_2 - x_3 = 0$ in every row.

Caution

Rank deficiency does not make your *predictions* wrong. It makes your *coefficients* non-unique, and with them every standard error, t-statistic and p-value. A library that silently returns one of infinitely many answers is doing you no favours: check `np.linalg.matrix_rank(X)` against `X.shape[1]` before you interpret anything.

3.8 Failure 2, measured

```
idx = rng.choice(len(X), size=8, replace=False)
Xw = np.c_[np.ones(8), X[idx]] # 8 rows, 11 columns
bmin, *_ = np.linalg.lstsq(Xw, y[idx], rcond=None)
ns = np.linalg.svd(Xw)[2][np.linalg.matrix_rank(Xw):] # the null space
```

```
8 patients, 10 features + intercept -> X is (8, 11)
rank(X) = 8, so rank(X'X) = 8 but X'X is 11 x 11
np.linalg.inv  ->  LinAlgError: Singular matrix
lstsq minimum-norm solution ||b|| = 31.5783, SSE = 3.709e-24
add 100 * a null-space direction: ||b|| = 104.8675, SSE = 2.614e-23
both fit the 8 rows exactly; they are two of infinitely many answers
```

The null space of X has dimension $11 - 8 = 3$, so you may move $\hat{\beta}$ anywhere in a three-dimensional affine set at exactly zero cost in SSE. Both vectors above fit all eight patients

perfectly, which is itself the warning: a model that reproduces the training data exactly has learned the data, not the process.

Key idea

The disease in the wide case is not "too little signal". It is **no unique answer**. Ridge regression replaces $X^\top X$ by $X^\top X + \lambda I$, which is invertible for every $\lambda > 0$, and thereby selects one member of that infinite family. That is the whole motivation for the next lecture.

3.9 The geometry: residuals are orthogonal to the predictors

Rearranging the normal equations (10) gives a statement worth memorising:

$$X^\top (y - X\hat{\beta}) = X^\top e = 0. \tag{12}$$

Every column of X is orthogonal to the residual vector. Equivalently, $\hat{y} = X\hat{\beta}$ is the orthogonal projection of y onto the column space of X, and e is the component of y that no linear combination of the predictors can reach. Writing $H = X(X^\top X)^{-1}X^\top$ for the **hat matrix**, $\hat{y} = Hy$ and $e = (I - H)y$; H is symmetric, $H^2 = H$, and $\operatorname{tr}(H) = p + 1$.

Two consequences follow from (12) and are worth stating separately, because students are regularly impressed by them when they should not be.

1. The first column of X is all ones, so $\sum_i e_i = 0$: **the residuals always sum to zero.**
2. The jth column is x_j, so $\sum_i x_{ij} e_i = 0$: **the residuals are uncorrelated with every predictor, and hence with $\hat{y}$.**

Caution

"My residuals have mean zero and are uncorrelated with the fitted values" is not evidence that the model is correct. It is arithmetic, true of every least squares fit ever computed, including a straight line fitted to a parabola. Section 9.2 shows exactly that case.

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.;Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.;Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.;Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.

4 The five flats, worked completely

This section does the whole calculation by hand, in integers, and then checks it against the library. The numbers were chosen so that nothing rounds.

4.1 Step 1: form $X^\top X$ and $X^\top y$

Six sums and three more

$$X^\top X = \begin{pmatrix} n & \sum x_1 & \sum x_2 \\ \sum x_1 & \sum x_1^2 & \sum x_1 x_2 \\ \sum x_2 & \sum x_1 x_2 & \sum x_2^2 \end{pmatrix}, \qquad X^\top y = \begin{pmatrix} \sum y \\ \sum x_1 y \\ \sum x_2 y \end{pmatrix}.$$

$\sum x_1 = 1+2+3+4+5 = 15$	$\sum y = 8+13+15+11+12 = 59$
$\sum x_2 = 2+3+4+3+4 = 16$	$\sum x_1 y = 8+26+45+44+60 = 183$
$\sum x_1^2 = 1+4+9+16+25 = 55$	$\sum x_2 y = 16+39+60+33+48 = 196$
$\sum x_2^2 = 4+9+16+9+16 = 54$	
$\sum x_1 x_2 = 2+6+12+12+20 = 52$	

$$X^\top X = \begin{pmatrix} 5 & 15 & 16 \\ 15 & 55 & 52 \\ 16 & 52 & 54 \end{pmatrix}, \qquad X^\top y = \begin{pmatrix} 59 \\ 183 \\ 196 \end{pmatrix}.$$

$X^\top X$ is symmetric — it always is — so only six of its nine entries are separate computations.

4.2 Step 2: invert $X^\top X$

Use cofactors. For a symmetric 3×3 matrix the adjugate is symmetric too, so there are six cofactors to compute, not nine.

$$\begin{aligned} A_{11} &= 55 \cdot 54 - 52^2 = 266, & A_{22} &= 5 \cdot 54 - 16^2 = 14, \\ A_{12} &= -(15 \cdot 54 - 52 \cdot 16) = 22, & A_{23} &= -(5 \cdot 52 - 15 \cdot 16) = -20, \\ A_{13} &= 15 \cdot 52 - 55 \cdot 16 = -100, & A_{33} &= 5 \cdot 55 - 15^2 = 50. \end{aligned}$$

Expanding along the first row,

$$\det(X^\top X) = 5(266) + 15(22) + 16(-100) = 1330 + 330 - 1600 = 60,$$

and therefore

$$(X^\top X)^{-1} = \frac{1}{60} \begin{pmatrix} 266 & 22 & -100 \\ 22 & 14 & -20 \\ -100 & -20 & 50 \end{pmatrix}. \tag{13}$$

4.3 Step 3: multiply out

$\hat\beta = (X^\top X)^{-1} X^\top y$, in integers

$$\begin{aligned} 60\, b_0 &= 266(59) + 22(183) - 100(196) = 15694 + 4026 - 19600 = 120, \\ 60\, b_1 &= 22(59) + 14(183) - 20(196) = 1298 + 2562 - 3920 = -60, \\ 60\, b_2 &= -100(59) - 20(183) + 50(196) = -5900 - 3660 + 9800 = 240. \end{aligned}$$

Dividing by 60:

$$\hat\beta = (b_0, b_1, b_2) = (2, -1, 4), \qquad \text{that is} \qquad \hat y = 2 - x_1 + 4x_2.$$

The bedroom coefficient is negative. An extra $50\,\mathrm{m}^2$ of floor area is worth $+4$ lakh; an extra bedroom *within the same floor area* is worth -1 lakh. That is precisely what flats B and D told us before any arithmetic was done.

4.4 Step 4: check the fit

	A	B	C	D	E
$\hat y = 2 - x_1 + 4x_2$	9	12	15	10	13
y	8	13	15	11	12
residual e	-1	$+1$	0	$+1$	-1

Now apply the orthogonality condition (12) as a check:

$$\sum e_i = 0, \qquad \sum x_{1i} e_i = -1 + 2 + 0 + 4 - 5 = 0, \qquad \sum x_{2i} e_i = -2 + 3 + 0 + 3 - 4 = 0.$$

All three are zero, so the coefficients satisfy the normal equations exactly.

The fit statistics follow. $\text{SSE} = 1+1+0+1+1 = 4$; $\bar{y} = 11.8$ and $\text{SST} = 3.8^2 + 1.2^2 + 3.2^2 + 0.8^2 + 0.2^2 = 26.8$, so

$$R^2 = 1 - \frac{4}{26.8} = 0.8507, \qquad R^2_{\text{adj}} = 1 - \frac{\text{SSE}/(n-p-1)}{\text{SST}/(n-1)} = 1 - \frac{4/2}{26.8/4} = 0.7015.$$

Key idea

$X^\top e = 0$ is a free correctness check on any hand computation of $\hat{\beta}$. It costs two minutes and it catches every arithmetic slip. Do it every time.

4.5 The same thing in code

```
X = np.c_[np.ones(5), x1, x2]
beta = np.linalg.inv(X.T @ X) @ (X.T @ y) # by hand
lr = LinearRegression().fit(np.c_[x1, x2], y) # by library
```

```
X'X =
 [[ 5 15 16]
 [15 55 52]
 [16 52 54]]
X'y = [ 59 183 196]
det(X'X) = 60
60 * inv(X'X) =
 [[ 266.   22. -100.]
 [  22.   14.  -20.]
 [-100.  -20.   50.]]
beta = inv(X'X) X'y = [ 2. -1.  4.]
LinearRegression intercept 2.0000000000  coef [-1.  4.]
max |difference| 8.62e-14
```

```
fit = X @ beta
r = y - fit
print(X.T @ r) # the normal equations, rearranged
```

```
fitted    [ 9. 12. 15. 10. 13.]
residuals [-1.  1.  0.  1. -1.]
X' r = [-0. -0. -0.]  (the normal equations, rearranged)
SSE 4.0000   SST 26.8000   R2 0.850746
adjusted R2 = 1 - (SSE/(n-p-1)) / (SST/(n-1)) = 0.701493
sklearn .score() 0.850746
```

Every digit of the hand calculation, reproduced. The 8.62×10^{-14} is floating point, not disagreement.

Caution

Do not write `np.linalg.inv(X.T @ X) @ X.T @ y` in production code. Forming the inverse squares the condition number of X and is both slower and less accurate than solving the system directly. `np.linalg.lstsq(X, y)` — which is what `LinearRegression` calls — uses an SVD and never forms $X^\top X$ at all. The explicit inverse is for understanding, not for use.

4.6 A shortcut for hand calculation: centre first

If you subtract the column means before fitting, the intercept separates out and you need only invert a $p \times p$ matrix instead of $(p+1) \times (p+1)$. Writing $S_{jk} = \sum_i (x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)$ and

$S_{jy} = \sum_i (x_{ij} - \bar{x}_j)(y_i - \bar{y})$,

$$\begin{pmatrix} S_{11} & S_{12} \\ S_{12} & S_{22} \end{pmatrix} \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} = \begin{pmatrix} S_{1y} \\ S_{2y} \end{pmatrix}, \qquad b_0 = \bar{y} - b_1\bar{x}_1 - b_2\bar{x}_2. \tag{14}$$

```
centred cross-product matrix
 [[10.   4. ]
 [ 4.   2.8]]
det 12.0   inverse * 12 =
 [[ 2.8 -4. ]
 [-4.  10. ]]
X'y centred       [6.0 7.2]
slopes from the 2x2 system [-1.  4.]
intercept = ybar - b1*x1bar - b2*x2bar = 2.0
```

By hand: the determinant is $10(2.8) - 4^2 = 12$, so

$$b_1 = \frac{2.8(6.0) - 4(7.2)}{12} = \frac{-12}{12} = -1, \qquad b_2 = \frac{-4(6.0) + 10(7.2)}{12} = \frac{48}{12} = 4,$$

and $b_0 = 11.8 - (-1)(3) - 4(3.2) = 11.8 + 3 - 12.8 = 2$. Two lines of arithmetic. **This is the form to use under examination conditions.**

Try it yourself

Refit the five flats after *deleting* flat C, whose residual is zero. The deleted point lies exactly on the fitted plane, so you might expect nothing to change. Compute $X^\top X$, det, and $\hat{\beta}$ for the remaining four rows. You should find that $\hat{\beta}$ is unchanged at $(2, -1, 4)$ — deleting a point with zero residual does not move a least-squares fit — while $\det(X^\top X)$ drops from 60 to 12 and R^2 changes because SST changes.

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.;Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.;Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.;Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.

5 Interpreting the coefficients

5.1 "Holding the others fixed"

Key idea

b_j is the change in $\hat{y}$ associated with a one-unit increase in x_j **when every other predictor in the model is held constant**. The clause is not decoration. It names the other columns you happened to include, and it is the reason b_j changes when you change them.

Three things follow immediately, and each of them catches people out.

The coefficient belongs to the model, not to the column. There is no such thing as "the effect of x_j". There is the effect of x_j in *this* model, with *these* other predictors. Add a column and every coefficient may move.

The clause may describe an impossible intervention. "Holding total cholesterol fixed, LDL cholesterol has effect b" describes a manipulation nobody can perform, since LDL is a component of the total. The number is still well defined and still computable. It simply does not mean what a newspaper headline would say it means.



Figure 1: Left: price against bedrooms. The orange line, which ignores floor area, has slope +0.6. The dashed segments join the pairs of flats that *share* a floor area, and they go down: −1.0 per bedroom at 150 m^2, −1.5 at 200 m^2. Right: the fitted plane $\hat{y} = 2 - x_1 + 4x_2$, with the five residuals drawn as vertical bars whose squares sum to SSE = 4.

The sign can be the opposite of the marginal association. This is the subject of the next subsection, and it is the single most important thing in this lecture.

For the five flats, $b_1 = -1$ does not say that bedrooms make a flat cheaper. It says: among flats of the *same floor area*, the one chopped into more bedrooms is worth less — smaller rooms. Meanwhile the marginal slope +0.6 says: flats with more bedrooms tend to be bigger, and bigger flats cost more. Both are true, and they are answers to different questions.

5.2 The sign flip, on real data

```
full = LinearRegression().fit(X, y)
for j, name in enumerate(dia.feature_names):
    simple = LinearRegression().fit(X[:, [j]], y).coef_[0]
    print(name, simple, full.coef_[j])
```

```
feature   simple slope   slope in the full 10-feature model
age           +1.1050                -0.0364  <-- SIGN FLIP
sex           +6.6454               -22.8596  <-- SIGN FLIP
bmi          +10.2331                +5.6030
bp            +2.4607                +1.1168
s1            +0.4723                -1.0900  <-- SIGN FLIP
s2            +0.4412                +0.7465
```

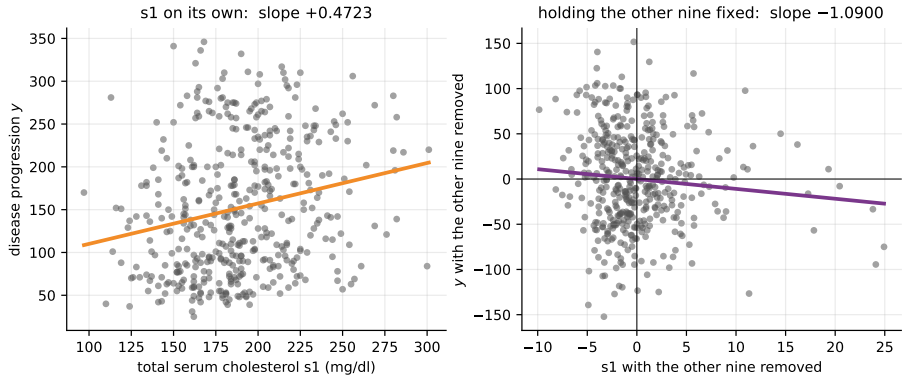


Figure 2: Left: raw `s1` against disease progression for all 442 patients, with the simple regression line of slope +0.4723. Right: the *added-variable plot* for `s1`. Both axes have had the other nine predictors regressed out, and the remaining relationship slopes *down*, at exactly the multiple-regression coefficient −1.0900. Same patients, same column, opposite conclusion.

```
s3              -2.3531               +0.3720  <-- SIGN FLIP
s4             +25.7158               +6.5338
s5             +83.5114              +68.4831
s6              +2.5649               +0.2801
```

Four of the ten predictors change sign. Take `s1`, total serum cholesterol. On its own its slope is +0.4723 per mg/dl: more cholesterol, worse disease progression, which is what everybody expects. Put the other nine measurements alongside it and the slope becomes −1.0900: the opposite sign, and 2.31 times the magnitude.

Caution

Ten simple regressions are not one multiple regression, and no amount of care in running them makes them one. A simple slope absorbs the effect of everything correlated with that column; a multiple slope has had those effects removed. Reporting a table of simple slopes and describing it as "the effect of each variable" is the commonest error in applied regression, and it is wrong by more than a factor of two here, in the wrong direction.

5.3 Where the difference goes: the omitted-variable formula

Suppose the true model has two predictors but you fit only the first. Write b_1, b_2 for the coefficients in the two-predictor fit and δ for the slope of x_2 regressed on x_1. Then

$$b_1^{\text{simple}} = b_1 + b_2\,\delta. \tag{15}$$

In words: the simple slope is the partial slope *plus* whatever effect x_1 borrows from x_2 by being correlated with it. The borrowed term is large when x_2 matters a lot (b_2 large) and when the two predictors move together (δ large).

```
pair = LinearRegression().fit(X[:, [i_s1, i_s5]], y)
delta = LinearRegression().fit(X[:, [i_s1]], X[:, i_s5]).coef_[0]
print(pair.coef_[0] + pair.coef_[1] * delta)
```

```
s1 (total cholesterol) alone            +0.4723
s1 with s5 (log triglycerides) added    -0.2418
s5 in that pair                         +91.7683
corr(s1, s5) +0.5155    corr(s1, y) +0.2120    corr(s5, y) +0.5659

the omitted-variable formula: b_simple = b_partial + b_s5 * delta
   regress s5 on s1: delta = +0.007781
   -0.2418 + +91.7683 * +0.007781 = +0.4723   (simple slope +0.4723)
```

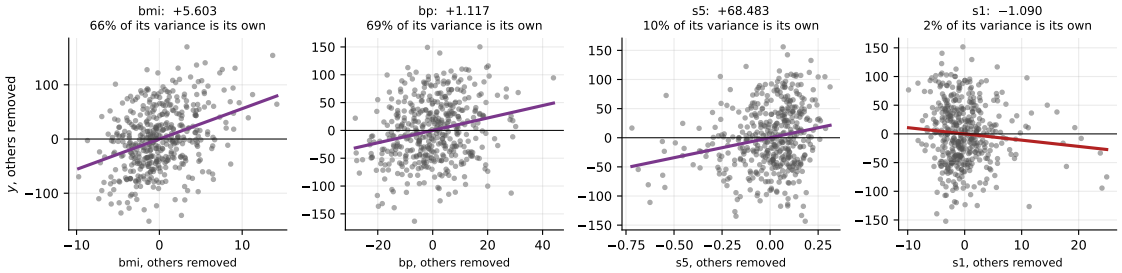


Figure 3: Added-variable plots for four of the ten diabetes predictors. The slope of each cloud *is* the coefficient in the ten-predictor model. The second line of each title is the fraction of that predictor's variance that survives regression on the other nine: 66% for `bmi`, 69% for `bp`, 9.9% for `s5` and only 1.7% for `s1`. The narrower the horizontal spread, the less the data has to say about that coefficient.

The identity holds to four decimal places, as it must — it is algebra, not an approximation. The borrowed term is $91.7683 \times 0.007781 = 0.714$, and $-0.2418 + 0.714 = +0.4723$. **Two predictors are enough to flip the sign; adding the other eight takes it from -0.24 to -1.09.**

Key idea

A coefficient changes when you add a predictor if and only if the new predictor is correlated with the old one *and* with the target. If either correlation is zero, nothing moves. This is why designed experiments randomise — randomisation makes $\delta = 0$ in expectation and the simple slope is then already the partial one.

5.4 Partial regression coefficients and added-variable plots

There is a second, constructive definition of b_j that never mentions the other coefficients:

1. Regress x_j on all the *other* predictors and keep the residual $\tilde{x}_j$. This is the part of x_j that nothing else in the model explains.
2. Regress y on all the other predictors and keep the residual $\tilde{y}$.
3. Regress $\tilde{y}$ on $\tilde{x}_j$. The single slope you get is b_j — exactly, not approximately.

This is the **Frisch–Waugh–Lovell** theorem, and the quantity it produces is what the syllabus calls the **partial regression coefficient**. It is the precise content of "holding the others fixed": both variables have had the other predictors' influence subtracted before they are compared.

```
others = [k for k in range(X.shape[1]) if k != j]
ex = X[:, j] - LinearRegression().fit(X[:, others], X[:, j]).predict(X[:, others])
ey = y - LinearRegression().fit(X[:, others], y ).predict(X[:, others])
print(LinearRegression().fit(ex.reshape(-1, 1), ey).coef_[0])
```

```
residual of s1 after regressing it on the other nine   sd 4.4979
residual of y  after regressing it on the other nine   sd 53.7607
slope of (y residual) on (s1 residual)  -1.0899963341
coefficient of s1 in the full model     -1.0899963341
difference 2.15e-14

corr(s1 residual, s1) 0.1300  -- only 1.7% of s1's variance is its own
```

The scatter of $\tilde{y}$ against $\tilde{x}_j$ is the **added-variable plot** (also called a partial regression plot). It is the only honest two-dimensional picture of a multiple-regression coefficient, and it tells you three things at once: the slope is b_j; the horizontal spread shows how much independent information x_j carries; and outliers and curvature that the raw scatter hides become visible.

Key idea

Only 1.7% of `s1`'s variance is its own. Its coefficient is estimated from that 1.7% and from nothing else. That single number predicts everything in Section 7.

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.;Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.;Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.;Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.

6 Standardized (beta) coefficients

6.1 Why raw coefficients cannot be compared

b_j has units of y per unit of x_j. Two coefficients whose denominators are different units are not the same kind of number, and ranking them by magnitude ranks the units, not the predictors. In the diabetes model the largest raw coefficient is `s5` = +68.48 — but `s5` is a logarithm with a standard deviation of 0.52, while `s1` is in mg/dl with a standard deviation of 34.6. A one-unit change means something completely different in the two columns.

The fix is to put both sides in standard-deviation units. Replacing x_j by $(x_j - \bar{x}_j)/s_j$ divides that column by s_j and therefore multiplies its coefficient by s_j; replacing y by $(y-\bar{y})/s_y$ divides every coefficient by s_y. So

$$\beta_j^* = b_j \frac{s_j}{s_y}, \tag{16}$$

the **standardized** or **beta** coefficient: the change in y, in standard deviations of y, per one standard deviation of x_j, holding the others fixed. It is dimensionless, and therefore comparable across predictors.

6.2 Measured, and the ranking changes

```
sd = X.std(axis=0, ddof=1)
beta_star = full.coef_ * sd / y.std(ddof=1)
```

```
feature    raw coef        sd      beta*   rank raw   rank beta*
s1           -1.0900   34.6081    -0.4893        6            1
s5          +68.4831    0.5224    +0.4640        1            2
bmi          +5.6030    4.4181    +0.3211        4            3
s2           +0.7465   30.4131    +0.2945        7            4
bp           +1.1168   13.8313    +0.2004        5            5
sex         -22.8596    0.4996    -0.1481        2            6
s4           +6.5338    1.2904    +0.1094        3            7
s3           +0.3720   12.9342    +0.0624        8            8
s6           +0.2801   11.4963    +0.0418        9            9
age          -0.0364   13.1090    -0.0062       10           10

largest by raw magnitude  : s5 (+68.4831)
largest by beta*          : s1 (-0.4893)
```

`s1` moves from sixth by raw magnitude to **first** by β^*; `sex` moves from second to sixth. The raw ranking was a ranking of measurement units.

The unit-free claim is checkable. Record `bmi` in units of 10^{-4} kg/m^2 — the same data, multiplied by 10^4 — and refit:

```
   bmi coefficient  +5.6030  (kg/m2)   +0.00056030  (1e-4 kg/m2)
   beta* for bmi    +0.321100  and  +0.321100   -- unchanged
check against a regression on standardised columns: max diff 7.72e-15
```

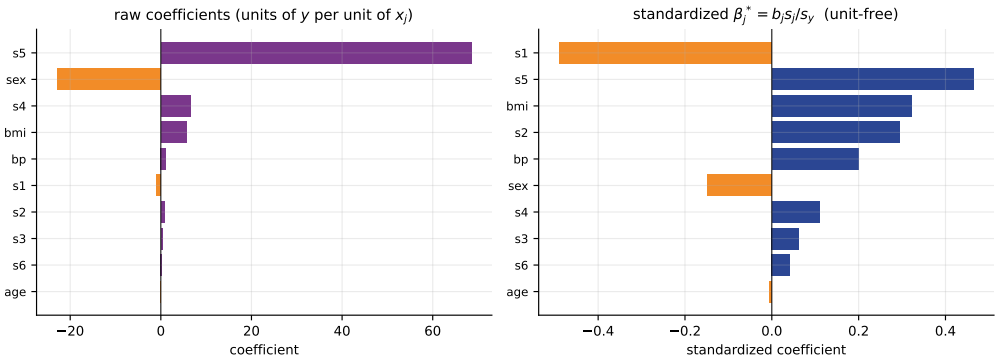


Figure 4: Left: the raw coefficients of the ten-predictor diabetes model, sorted by magnitude. One bar dwarfs the rest and it tells you only that `s5` is recorded in small numbers. Right: the same fit, standardized. Five predictors now sit within a factor of two of each other and `s1` has come first.

The raw coefficient changed by a factor of 10^4; β^* did not change at all. And rescaling by s_j/s_y gives exactly the same numbers as putting a `StandardScaler` in front of the regression, to 7.7×10^{-15}.

Caution

β^* solves the units problem and nothing else. It is still conditional on the other predictors; it still inherits any instability from multicollinearity; and s_j is estimated from *this* sample, so a predictor with an unusually wide spread in your data will look important. β^* ranks; it does not adjudicate, and it certainly does not establish causation.

Try it yourself

Fit the wine table: predict `proline` from `alcohol`, `flavanoids` and `color_intensity`. You should get $R^2 = 0.5471$, adjusted $R^2 = 0.5393$, raw coefficients $(+187.07, +126.41, +16.54)$ and $\beta^* = (0.4823, 0.4010, 0.1217)$. Note that here the raw and standardized rankings *agree* — the three columns happen to have comparable spreads. Standardizing is not always dramatic; it is always safe.

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.;Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.;Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.;Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.

7 Multicollinearity

7.1 Near-dependence, and what it costs

Section 3.6 dealt with *exact* dependence, where $X^\top X$ is singular. The practically important case is *near* dependence: $X^\top X$ is invertible, so everything runs, but the inverse is enormous and the coefficients are estimated from very little information.

The relevant quantity is how well each predictor can be reconstructed from the others. Let R_j^2 be the R^2 from regressing x_j on all the other predictors. Then the variance of the estimated coefficient is

$$\operatorname{Var}(\hat{\beta}_j) = \frac{\sigma^2}{(n-1)s_j^2} \cdot \frac{1}{1-R_j^2}, \qquad \mathrm{VIF}_j = \frac{1}{1-R_j^2}. \tag{17}$$

The first factor is what you would get if the predictors were orthogonal. The second, the **variance inflation factor**, is the price of collinearity. Standard errors — which are square roots of variances — are multiplied by $\sqrt{\mathrm{VIF}_j}$.

Reading a VIF

$R_j^2 = 0$ gives VIF $= 1$: no inflation. $R_j^2 = 0.5$ gives 2. $R_j^2 = 0.9$ gives 10, so a standard error $\sqrt{10} = 3.2$ times larger. $R_j^2 = 0.99$ gives 100 and a standard error ten times larger — which means you need 100 times as many observations to estimate that coefficient as precisely as an uncorrelated one.

The conventional thresholds are VIF > 5 "look into it" and VIF > 10 "there is a problem". They are conventions, not theorems. What matters is whether the coefficient you intend to *interpret* is one of the inflated ones.

```
def vif(A):
    out = []
    for j in range(A.shape[1]):
        o = [k for k in range(A.shape[1]) if k != j]
        rj2 = LinearRegression().fit(A[:, o], A[:, j]).score(A[:, o], A[:, j])
        out.append(1.0 / (1.0 - rj2))
    return np.array(out)
```

```
feature      R2_j      VIF   sqrt(VIF)
s1         0.9831    59.20     7.69
s2         0.9745    39.19     6.26
s3         0.9351    15.40     3.92
s5         0.9008    10.08     3.17
s4         0.8875     8.89     2.98
bmi        0.3375     1.51     1.23
s6         0.3264     1.48     1.22
bp         0.3148     1.46     1.21
sex        0.2176     1.28     1.13
age        0.1785     1.22     1.10

rule of thumb: VIF > 5 is worth a look, VIF > 10 is a problem
here 4 of the 10 exceed 10

drop s1 (the worst) and refit:
   s4       VIF     7.82
   s3       VIF     3.74
   s2       VIF     2.93
   s5       VIF     2.17
```

98.31% of `s1` is predictable from the other nine columns — which is the same fact as "only 1.7% of its variance is its own" from Section 5.4, seen from the other side. Its standard error is 7.69 times what it would be if the columns were orthogonal. And the structure is no mystery: `s4` is `s1`/`s3` by definition and `s2` is part of `s1`. Dropping `s1` alone takes the worst remaining VIF from 59.2 to 7.8.

7.2 Coefficient instability, measured by the bootstrap

A VIF is a prediction about variance. The bootstrap measures it directly: resample the 442 patients with replacement, refit, and look at the spread of the coefficients across 2000 resamples. Each resample is a plausible alternative dataset from the same population.

```
BOOTSTRAPS = range(0, 2000) # a sweep: 2000 resamples, one seed each,
                            # derived from the lecture seed 0
for b, bseed in enumerate(BOOTSTRAPS):
    s = np.random.default_rng(bseed).integers(0, len(X), len(X))
    m = LinearRegression().fit(X[s], y[s])
    coef[b] = m.coef_
    pred[b] = m.predict(X.mean(axis=0).reshape(1, -1))[0]
```

```
2000 bootstrap resamples of the 442 patients
```

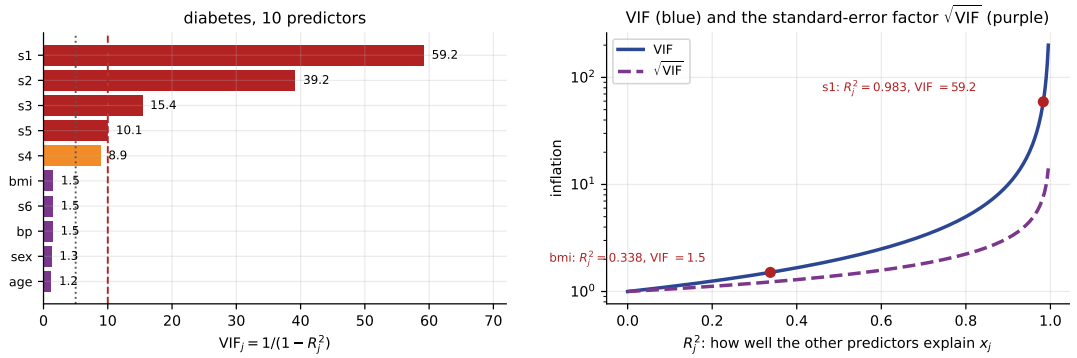


Figure 5: Left: the ten variance inflation factors, with the conventional thresholds at 5 (dotted) and 10 (dashed). Right: the curve $\mathrm{VIF} = 1/(1 - R_j^2)$ on a logarithmic axis, together with the standard-error factor $\sqrt{\mathrm{VIF}}$. The curve is nearly flat until $R_j^2 \approx 0.8$ and then goes vertical, which is why collinearity tends to arrive suddenly rather than gradually.

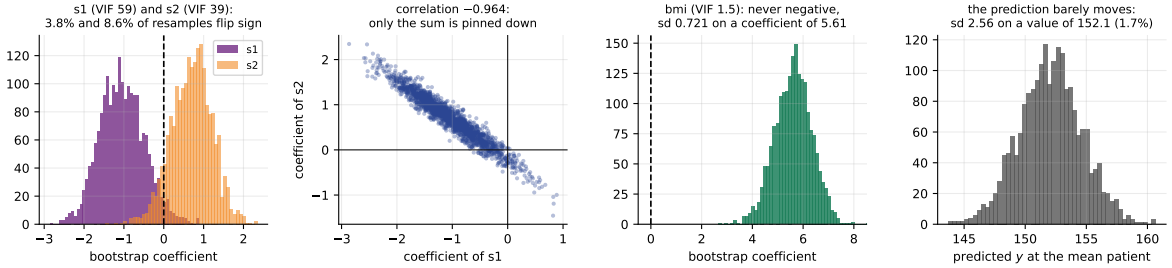


Figure 6: 2000 bootstrap resamples. Panels 1 and 2: the `s1` and `s2` coefficients are individually uncertain and jointly almost perfectly anti-correlated — their sum is pinned down and the split is not. Panel 3: `bmi`, with VIF = 1.5, is never negative. Panel 4: the prediction at the mean patient has a standard deviation of 1.7% of its value, while the coefficients producing it are barely determined.

```
feature     coef     boot sd     2.5%      97.5%     VIF   sign flips
s1         -1.090     0.565    -2.093     +0.113    59.2       3.8%
s2         +0.746     0.512    -0.325     +1.634    39.2       8.6%
s5        +68.483    14.982   +39.101    +97.225    10.1       0.0%
bmi        +5.603     0.721    +4.159     +6.996     1.5       0.0%
bp         +1.117     0.221    +0.680     +1.545     1.5       0.0%

correlation between the bootstrap s1 and s2 coefficients: -0.9637

prediction at the mean patient
   full 10-feature model  mean 152.0573  boot sd 2.5635
   with s1 dropped        mean 152.0647  boot sd 2.5818
   bmi coefficient        boot sd 0.7212 (full) vs 0.7374 (s1 dropped)
```

Read the intervals. `bmi`, with VIF = 1.5, has a 95% bootstrap interval of $[+4.16, +7.00]$: nowhere near zero, and its sign never changed in 2000 resamples. `s1` has $[-2.09, +0.11]$ and `s2` has $[-0.33, +1.63]$: **both straddle zero**. The fitted `s2` coefficient is positive, yet it came out *negative* in one resample out of twelve; the fitted `s1` coefficient is negative, yet it came out positive in about one in twenty-six. Neither sign is established by 442 patients.

The correlation of -0.964 between the two bootstrap coefficients is the mechanism made visible. `s1` and `s2` are nearly the same column, so the data determines their *sum* well and the split between them hardly at all. Every resample slides the pair along a line.

7.3 Predicting is fine. Explaining is not.

This is the distinction the whole section exists for. Take two predictors correlated at 0.9986, generated so that the truth is $y = 3x_1 + 2x_2 + \varepsilon$. Fit on the first hundred rows, then on the

second hundred.

```
corr(a, b) 0.9986    true coefficients 3.0 and 2.0     VIF [351.6 351.6]
fitted on all 200 rows: +2.438  +2.595  (sum +5.034, truth 5.000)
first half   a   +2.553  b   +2.460   sum  +5.012   test R2 +0.9895
second half  a   +2.208  b   +2.848   sum  +5.056   test R2 +0.9895
the two coefficient vectors differ by 0.39, their sum by 0.044
```

Read the individual coefficients against the truth. The generating values were $b_1 = 3$ and $b_2 = 2$, and **not one of the three fits recovers them**: on all 200 rows least squares reports $(+2.44, +2.60)$, and the two halves report $(+2.55, +2.46)$ and $(+2.21, +2.85)$. The estimates wander over a range of about 0.4 from one half of the data to the other, and they sit roughly half a unit from the truth in opposite directions.

Now read the sum. It is 5.034, 5.012 and 5.056 against a truth of 5.000 — **agreeing to about four parts in a thousand while the individual coefficients are simply wrong**. Both halves score $R^2 = 0.9895$ on the same evaluation data, identical to four decimal places, and their predictions correlate at $r = 0.99999$.

That is the whole lesson in one table: what the data constrains here is $b_1 + b_2$, and the split between the two is decided by noise. Be careful not to over-claim it. On this particular sample the two halves do *not* disagree about the *sign* of b_2 — both are positive. Collinearity does not guarantee a sign flip; it guarantees that the split is unreliable, and a sign flip is simply what that unreliability looks like when the coefficient is close enough to zero. For sign flips that really did happen, look back at the real diabetes table in Section 5.2, where four of ten coefficients change sign, and at the bootstrap above, where `s2` flips in 8.6% of resamples.

The same point on the real table:

```
cross-validated R2, all 10 features        +0.4892
cross-validated R2, s1 dropped             +0.4867
cross-validated R2, s1 and s2 both dropped +0.4867
cross-validated R2, ridge(alpha=1) on standardised columns +0.4896

ridge coefficients are stable where least squares is not:
   feature   OLS (standardised)   ridge
   s1               -37.6800  -30.0884
   s2               +22.6762  +16.6532
   s5               +35.7344  +32.8438
   bmi              +24.7265  +24.7712
```

Dropping the two worst-behaved predictors costs 0.0025 of cross-validated R^2 — nothing.

Key idea

Multicollinearity is a disease of the coefficients, not of the predictions. If all you need is $\hat{y}$, you may ignore it entirely. If anybody is going to read a coefficient, write it in a report, or make a decision from its sign, you may not.

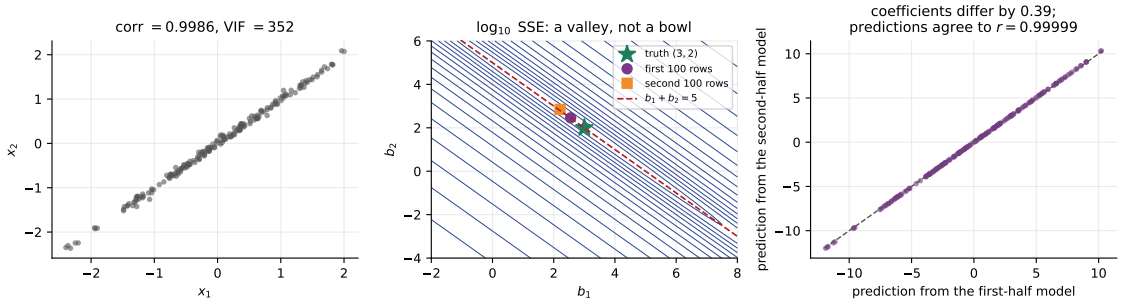


Figure 7: Left: two predictors correlated at 0.9986, VIF = 352. Middle: the residual sum of squares as a function of (b_1, b_2). It is a long trench along $b_1 + b_2 = 5$ rather than a bowl, so which point in the trench you land on is decided by noise; the purple circle and orange square are the two half-samples. Right: their predictions nevertheless agree to $r = 0.99999$.

7.4 What to do about it

	What it does	What it costs
Nothing	entirely acceptable if the model is only ever used to predict	no coefficient may be interpreted, and you must say so
Drop a column	removes the redundancy; VIFs fall immediately	the survivor now carries both effects, so the choice of which to keep is a scientific decision, not a statistical one
Combine columns	replace `s1` and `s2` by a sum, a ratio or an index	you must be able to name and defend the combination
Ridge or lasso	$X^\top X + \lambda I$ is invertible for every $\lambda > 0$, and the coefficients stabilise	the estimates are biased; λ must be chosen by cross-validation (next lecture)
More data	shrinks every standard error by $\sqrt{n}$	useless when the columns are collinear *by definition*, as `s4` = `s1`/`s3` is

Note what is *not* on that list: dropping a predictor because its p-value is large. In a collinear pair both p-values are large precisely because the two are fighting over the same information, and dropping either one makes the other significant.

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.;Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.;Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.;Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.

8 Missing data in a regression

8.1 Why it is worse here than you expect

Least squares needs a complete row. One missing cell in one column removes the entire observation from the fit, and with d columns each missing independently at rate p, a row survives with probability $(1 - p)^d$. That exponent is the whole problem.

p	rows kept of 309 ($d = 10$)	fraction
0.01	279	90.4%
0.02	252	81.7%
0.05	185	59.9%
0.10	108	34.9%
0.20	33	10.7%
0.30	9	2.8%

A 10% per-cell missing rate — which nobody would describe as a serious data quality problem — deletes two thirds of the patients, silently.

The earlier data-cleaning lecture introduced the three missingness mechanisms and they carry over unchanged. Under **MCAR** the complete cases are a fair random sample, so you lose precision but not correctness. Under **MAR** the complete cases are biased, but the observed columns contain enough information to undo it — which is exactly what a regression-based imputer does. Under **MNAR** the missingness depends on the unobserved value itself and no purely computational fix repairs it.

8.2 Five strategies, measured

309 training rows, ten predictors, MCAR missingness imposed on the training columns only, evaluated on a held-out third that is complete.

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import SimpleImputer, IterativeImputer
Pipeline([("imp", SimpleImputer(strategy="mean")),
          ("lr", LinearRegression())]).fit(X_missing, y_train).score(X_test, y_test)
```

```
no missingness at all                test R2 +0.3929

MCAR at 10%: cells missing 10.4%, complete rows 103 of 309 (33.3%)
   listwise deletion            103 rows   test R2 +0.3231
   mean imputation              309 rows   test R2 +0.3875
   median imputation            309 rows   test R2 +0.3850
   mean + missing indicators    309 rows   test R2 +0.3657
   IterativeImputer (MICE)      309 rows   test R2 +0.3871

MCAR at 25%: cells missing 25.2%, complete rows 13 of 309 (4.2%)
   listwise deletion             13 rows   test R2 -2.1451
   mean imputation              309 rows   test R2 +0.4048
   median imputation            309 rows   test R2 +0.3990
   mean + missing indicators    309 rows   test R2 +0.3526
   IterativeImputer (MICE)      309 rows   test R2 +0.4038
```

At 10% listwise deletion already loses 0.0698 of test R^2 against the complete table, while every imputer recovers most of it. At 25% listwise deletion leaves 13 rows to estimate 11 parameters and the result is $\mathbf{-2.15}$ — far worse than predicting $\bar{y}$ for every patient. Mean imputation, the crudest method available, scores +0.405 on the same data and `IterativeImputer` +0.404, both a little above the no-missingness baseline of +0.393.

Do not read anything into the ordering of those last two. The gap between mean imputation and MICE is 0.0010, and the gap between either of them and the complete-data baseline is +0.0119 and +0.0109, on a single split whose fold-to-fold spread elsewhere in this lecture is 0.11. **These differences are noise.** What the table does establish, unambiguously, is the size of the effect that matters: throwing rows away costs you 2.5380 of R^2, and any imputer at all costs you nothing measurable.

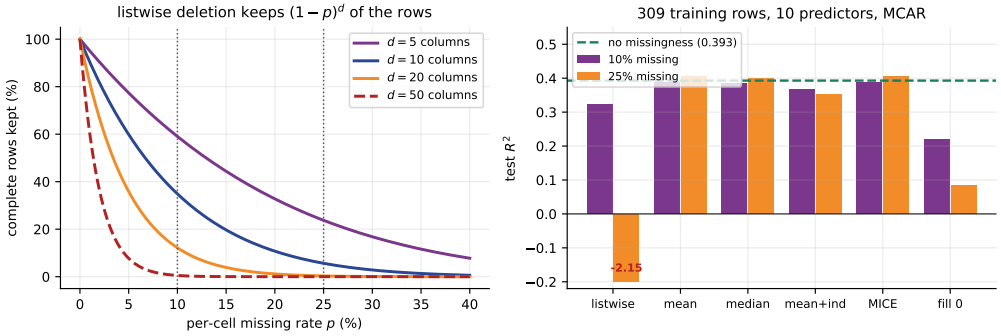


Figure 8: Left: the fraction of complete rows, $(1-p)^d$, for four widths of table. At $d = 50$ a 5% cell rate keeps about a fifth of the rows. Right: the five strategies at two missing rates against the no-missingness baseline. The listwise bar at 25% is off the bottom of the axis, at −2.15; "fill 0" is the other bad answer.

Caution

Filling missing values with 0 is not a neutral default. A patient with `age` = 0 and `bp` = 0 is not a patient with unknown age; it is a claim about a newborn with no blood pressure, and least squares believes it. Measured on the diabetes table at 25% missingness: cross-validated R^2 of +0.061, against +0.411 for mean imputation. Use `np.nan` and an imputer, never a sentinel.

8.3 Adding a missing-indicator column

Setting `add_indicator=True` on an imputer appends a binary column per predictor recording whether that cell was imputed. This is the right move when the *fact* of missingness is informative — a test that was not ordered because the patient looked healthy, a question a respondent declined. On this MCAR data it *hurt*: +0.3657 against +0.3875 at 10% missing, and +0.3526 against +0.4048 at 25%. That is exactly what you would expect once you think about it. Under MCAR the indicator carries no information about y at all, so each one is a pure-noise column — and the previous subsection is about what pure-noise columns do to a fit. Ten of them on 309 rows is enough to cost a couple of points of held-out R^2.

So: add indicators when you have a reason to believe the *fact* of missingness is informative, and not otherwise. Report what the experiment showed rather than the benefit you were hoping for.

8.4 An imputer is a learned step

An imputer estimates column means, or fits a regression per column. Those are numbers learned from data, so they belong inside the cross-validation fold.

```
# wrong: the column means are computed from every row, including test rows
X_filled = SimpleImputer().fit_transform(X)
cross_val_score(LinearRegression(), X_filled, y, cv=5)

# right: refitted on four fifths of the data inside every fold
cross_val_score(Pipeline([("imp", SimpleImputer()),
                          ("lr", LinearRegression())]), X, y, cv=5)
```

```
25% MCAR on all 442 rows, 5-fold cross-validation
   mean    impute-then-split +0.4114   inside the Pipeline +0.4112   difference +0.0002
   MICE    impute-then-split +0.4434   inside the Pipeline +0.4392   difference +0.0042
   an imputer never looks at y, so there is very little for it to leak
```

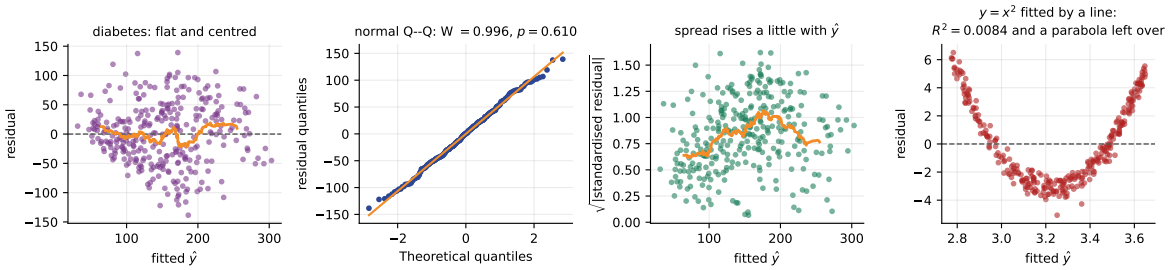


Figure 9: The first three panels are the diabetes fit: residuals against fitted (flat, with a smoothed trend in orange), a normal Q–Q plot, and a scale–location plot showing spread rising slightly with $\hat{y}$. The fourth panel is the point of the figure: $y = x^2 + \varepsilon$ fitted by a straight line. $R^2 = 0.0084$, the residual mean is zero, the residual–fitted correlation is zero — and the residual plot is a parabola.

Key idea

Report the size of a leak honestly. Here it is +0.0002 for mean imputation and +0.0042 for MICE — real but negligible, for the same reason that scaling and PCA leak very little: neither ever looks at y. Put the imputer in the `Pipeline` anyway. It costs nothing, and an imputer that *does* use the target, or a test set small enough for its mean to matter, is a different story.

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.;Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.;Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.;Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.

9 Model validation

9.1 Three questions, three checks

Question	Check	What failure looks like
Are the assumptions met?	residual plots, normal Q–Q, scale–location	curvature; a fan; heavy tails
Does it generalise?	train against test, or cross-validation	a large gap between them
Is this extra column earning its place?	adjusted R^2, held-out R^2, a partial F test	R^2 up while adjusted and held-out R^2 go down

9.2 Residual diagnostics

```
diabetes, 10 features, 309 training rows
   train R2 +0.5539   test R2 +0.3929
   train RMSE 52.954  test RMSE 55.652
   residual mean 6.88e-14 (least squares forces this to 0)
   corr(residual, fitted) -5.24e-16 (forced to 0 too)
   Shapiro-Wilk on the residuals: W 0.9959, p 0.6100
   Breusch-Pagan style check: corr(|residual|, fitted) +0.2079, p 0.0002
```

Take those lines one at a time. The residual mean and the residual–fitted correlation are zero *by construction* (Section 3.9) and carry no information. The Shapiro–Wilk test gives $p = 0.11$: no evidence against normal residuals. Train and test RMSE agree to within about 5% (52.95 against 55.65), which is the ordinary gap between fitting a sample and predicting a fresh one, not evidence of overfitting at 309×10. But $|e|$ rises with $\hat{y}$ at $p = 0.0002$: mild **heteroscedasticity**. The coefficients remain unbiased; the standard errors do not, and prediction intervals will be too narrow at the high end.

```
a model that is wrong in a way R2 will not tell you about:
   y = x^2 + noise, fitted with y = b0 + b1 x
   R2 0.0084   residual mean -4.32e-16   corr(residual, fitted) -8.79e-17
   but corr(residual, x^2) +0.9805  -- the plot shows a parabola
   add the x^2 column: R2 0.9710
```

Key idea

Every summary statistic on that fit is exactly what a correct model would produce, and the model is completely wrong. **Look at the residual plot.** Adding the one column the picture asks for takes R^2 from 0.0084 to 0.9710.

9.3 R^2 always rises; adjusted R^2 does not

Adding a column can never increase SSE, because the previous solution is still available with a coefficient of zero. Therefore

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}} \text{ is monotone non-decreasing in } p, \qquad \text{whatever the new column contains.} \quad (18)$$

R^2 is therefore useless for comparing models of different size. The standard correction compares mean squares rather than sums:

$$R^2_{\text{adj}} = 1 - \frac{\text{SSE}/(n-p-1)}{\text{SST}/(n-1)} = 1 - (1-R^2)\,\frac{n-1}{n-p-1}. \quad (19)$$

A useless column reduces SSE a little and reduces the denominator $n-p-1$ by one, so the ratio rises and R^2_{adj} falls. Unlike R^2 it can be negative.

The experiment: 80 training rows of diabetes, all ten real predictors, then k columns drawn from $\mathcal{N}(0,1)$ and independent of everything.

```
adding predictors that are REAL: bmi, then bp, s5, s4, s3 ...
 p   added     train R2   adj R2
 1   bmi          0.4706   0.4639
 2   bp           0.4751   0.4615
 3   s5           0.5481   0.5303
 4   s4           0.5496   0.5256
 5   s3           0.5667   0.5374
 6   s1           0.5751   0.5402

adding predictors that are NOISE: 80 training rows, 10 real features,
then k columns drawn from N(0,1) and independent of everything
 junk k      p   train R2   adj R2    test R2
      0     10     0.5911   0.5318     0.2488
      1     11     0.5992   0.5344     0.2295
      2     12     0.6048   0.5341     0.1939
      3     13     0.6061   0.5285     0.2003
      5     15     0.6183   0.5289     0.1719
      8     18     0.6226   0.5112     0.1571
     10     20     0.6407   0.5188     0.1340
     15     25     0.6502   0.4883     0.0756
     20     30     0.6935   0.5059    -0.0794

train R2 rises at every single step: 8 of 8 increases
adjusted R2 is highest at k = 1, i.e. it is fooled by the first 1 junk column(s) and then
    falls
test R2 falls at 7 of 8 steps, from +0.2488 to -0.0794
```

Both tables repay reading. With *real* predictors, R^2 and R^2_{adj} rise together — until `s1` joins at $p = 6$ and the adjustment falls, because `s1` is nearly redundant given `s3` and `s4`. With *noise*

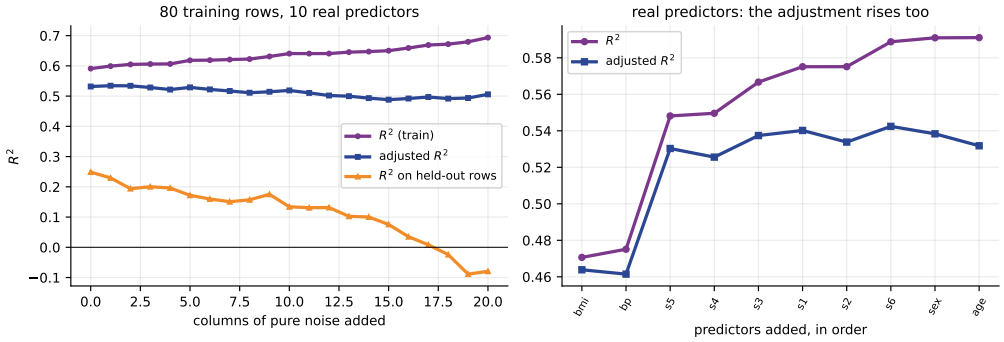


Figure 10: Left: as pure-noise columns are added to 80 training rows, R^2 only ever rises, adjusted R^2 drifts down and held-out R^2 collapses. Right: the same three-column arithmetic when the added predictors are real — both curves rise together, and the dip at `s1` and `s2` is the adjustment noticing two nearly redundant columns.

predictors, R^2 rose at all eight steps, and held-out R^2 fell at seven of the eight, from $+0.249$ all the way to $\mathbf{-0.079}$ — past zero, so the model with twenty junk columns is worse on fresh patients than predicting $\bar{y}$ for everybody.

Look hard at the middle column, because it is the one that catches people out. R^2_{adj} does *not* peak at no junk at all: it rises from 0.5318 to 0.5344 when the first junk column is added, and only then starts to fall. The adjustment is a fixed arithmetic penalty of $(n-1)/(n-p-1)$; it knows how many columns you used but nothing whatever about whether they were real, so a junk column that happens to fit a little noise can still pay for itself. **Adjusted R^2 can be fooled, and here it was.**

```
push it further: k = 60 gives 71 parameters for 80 rows
   train R2 0.9757   test R2 -7.3523   adjusted R2 0.7863
```

Caution

As $p \to n-1$ the factor $(n-1)/(n-p-1)$ explodes and R^2_{adj} stops meaning anything: at 71 parameters on 80 rows it reads a cheerful 0.786 while the held-out R^2 is -7.35. **Adjusted R^2 is a warning light, not a model selection criterion.** The criterion is held-out error.

9.4 Everything in one pipeline

```
pipe = Pipeline([
    ("imp", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()), # so the coefficients are beta*
    ("model", LinearRegression()),
])
cross_val_score(pipe, X, y, cv=KFold(5, shuffle=True, random_state=0))
```

```
impute -> standardise -> least squares, 5-fold on the 10%-missing table
   fold R2 [0.2632 0.3567 0.5213 0.3747 0.5293]
   mean +0.4090  sd 0.1143
   for comparison, the same pipeline on the complete table: +0.4892
```

The column medians and the column means and standard deviations are all recomputed inside every fold, from four fifths of the data. And read the fold-to-fold spread: 0.28 to 0.49. A single train/test split of 442 patients would have reported any number in that range with equal confidence, which is the argument for cross-validating rather than splitting once.

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.;Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.;Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.;Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.

10 Summary

1. $y = X\beta+\varepsilon$. Expanding $(y-Xb)^\top(y-Xb)$ and differentiating with the two rules $\partial(b^\top c)/\partial b = c$ and $\partial(b^\top Ab)/\partial b = 2Ab$ gives $\nabla S = -2X^\top y + 2X^\top Xb$, hence $X^\top X\hat\beta = X^\top y$ and $\hat\beta = (X^\top X)^{-1}X^\top y$. The Hessian $2X^\top X$ is positive semi-definite, so the stationary point is a global minimum.
2. $X^\top X$ is invertible exactly when X has full column rank, because $\text{null}(X^\top X) = \text{null}(X)$. It fails under perfect collinearity and when $p+1 > n$. In both cases `LinearRegression` returns a minimum-norm answer rather than raising, and that answer is one of infinitely many.
3. The five flats, in integers: $\det X^\top X = 60$, the adjugate has rows $(266, 22, -100)$, $(22, 14, -20)$ and $(-100, -20, 50)$, $\hat\beta = (2, -1, 4)$, residuals $(-1, 1, 0, 1, -1)$, $X^\top e = 0$, $R^2 = 0.8507$, $R^2_{\text{adj}} = 0.7015$ — and `LinearRegression` agrees to 8.6×10^{-14}.
4. b_j means "holding the others fixed". The bedroom coefficient is -1 because flats B and D have the same area and differ by two bedrooms and two lakh.
5. **Total serum cholesterol has slope** $+0.4723$ **alone and** -1.0900 **with nine other predictors in the model**, and four of the ten diabetes predictors change sign that way.
6. The omitted-variable identity accounts for it exactly: $-0.2418 + 91.7683 \times 0.007781 = +0.4723$. A coefficient moves when you add a predictor if and only if the new predictor is correlated with both the old one and the target.
7. Frisch–Waugh–Lovell: residualise x_j and y on the other predictors and the single slope between the residuals is b_j, to 2×10^{-14}. That scatter is the added-variable plot, and its horizontal spread tells you how much the data has to say.
8. Standardizing changes the ranking. `s1` goes from sixth by raw magnitude to **first** by $\beta^* = b_j s_j/s_y$, and β^* is unchanged when `bmi` is rescaled by 10^4.
9. $\text{VIF}_j = 1/(1 - R^2_j)$ multiplies $\text{Var}(\hat\beta_j)$. Four of the ten diabetes predictors exceed 10; `s1` reaches 59.2, so its standard error is 7.7× inflated, and only 1.7% of its variance is its own.
10. 2000 bootstrap resamples: the `s1` and `s2` coefficients both straddle zero and correlate at -0.964, while the prediction at the mean patient varies by 1.7%. On a collinear synthetic pair whose truth was $(3, 2)$, least squares returned $(+2.44, +2.60)$ and the two halves $(+2.55, +2.46)$ and $(+2.21, +2.85)$ — every one of them wrong — while the sum stayed within 0.06 of 5 and the two halves' predictions correlated at 0.99999. **Collinearity breaks explanation, not prediction.**
11. Missing data: $(1 - p)^d$. At 10% across ten columns, two thirds of rows are deleted; at 25%, listwise deletion scores -2.15 while mean imputation scores $+0.405$ and MICE $+0.404$ — a difference between those two that is well inside the noise. Filling with 0 scores $+0.061$. The imputer belongs in the `Pipeline`, though its leak here measured only $+0.0002$ to $+0.0042$.
12. R^2 rose at all eight steps as noise columns were added; adjusted R^2 was fooled by the first one and peaked there, not at zero junk; held-out R^2 fell from $+0.249$ to -0.079. At 71 parameters on 80 rows, adjusted R^2 read 0.786 and held-out R^2 was -7.35. **Only the held-out number is a criterion.** And a straight line fitted to a parabola has $R^2 = 0.0084$, residual mean zero and zero residual–fitted correlation: look at the plot.

11 Practice questions

Attempt these before reading Section 12. Questions 1, 2, 3, 4 and 6 are hand arithmetic and are the ones most likely to appear on a paper.

1. Starting from $S(b) = (y - Xb)^\top (y - Xb)$, derive the normal equations. State the two matrix-calculus rules you use and verify each by writing out one component. Then show that the stationary point is a minimum, and state precisely the condition under which the solution is unique.

2. A table has $n = 4$ rows and two predictors: $x_1 = (1, 2, 3, 4)$, $x_2 = (1, 3, 2, 4)$ and $y = (5, 15, 14, 20)$. (a) Form $X^\top X$ and $X^\top y$. (b) Compute $\det(X^\top X)$ and $(X^\top X)^{-1}$ by cofactors. (c) Obtain $\hat{\beta}$. (d) Verify your answer using $X^\top e = 0$. (e) Compute R^2 and R^2_{adj}.

3. Repeat question 2(c) using the centred 2×2 system of Equation (14). Which route would you use in an examination with $p = 3$, and why?

4. For each of the following design matrices, state whether $X^\top X$ is invertible and give the rank. (a) $n = 100$, $p = 3$, all columns distinct and no exact relation. (b) $n = 100$, columns "height in cm", "height in inches", "weight". (c) $n = 100$, a categorical variable with four levels one-hot encoded into four columns, plus an intercept. (d) $n = 12$, $p = 30$. For each case where it fails, give the fix and say what `LinearRegression` will do if you do not apply it.

5. A researcher regresses house price on the number of bedrooms and reports a slope of $+0.6$ lakh per bedroom. You then fit price on bedrooms and floor area and obtain -1 and $+4$. The slope of area on bedrooms is 0.4 (in units of $50\,\text{m}^2$ per bedroom). (a) Verify the two results using Equation (15). (b) Which number should appear in an advertisement for a flat conversion that adds a bedroom without extending the building, and why?

6. A model predicts monthly household spending in rupees from monthly income in rupees ($b = 0.42$, $s_j = 30{,}000$) and from household size in persons ($b = 3200$, $s_j = 1.8$). The standard deviation of spending is 18,000. (a) Compute both standardized coefficients. (b) Which predictor is more important, and what exactly does your answer mean? (c) The income column is re-expressed in thousands of rupees. What happens to b, to s_j and to β^*?

7. In a five-predictor model, regressing x_3 on the other four gives $R^2_3 = 0.96$. (a) Compute VIF_3 and the factor by which the standard error of $\hat{\beta}_3$ is inflated. (b) By what factor would n have to increase to recover the precision you would have had with orthogonal predictors? (c) The 95% bootstrap interval for $\hat{\beta}_3$ turns out to be $[-0.4, +2.1]$ while the model's cross-validated R^2 is a healthy 0.71. Write the two sentences you would put in the report.

8. A clinical table has $n = 500$ patients and $d = 12$ predictors, each missing independently at 8%. (a) How many complete rows do you expect? (b) You need 13 parameters — comment. (c) The missingness in one column is caused by clinicians not ordering an expensive test for patients who looked healthy. Classify the mechanism and say what it implies. (d) Give the pipeline you would actually run, and say why every step is inside it.

9. A colleague sends you this paragraph. "We fitted a linear model of hospital readmission on ten clinical variables. R^2 was 0.62, up from 0.55 with five variables, so the larger model is better. Total cholesterol had a coefficient of -1.09, so lowering cholesterol increases readmission risk; this contradicts the medical literature and is our most interesting finding. LDL cholesterol has the largest coefficient in the model at $+22.7$, so it is the strongest predictor. Residuals had mean zero and were uncorrelated with the fitted values, confirming the model is well specified." Identify every error, say what evidence you would demand for each claim, and rewrite the paragraph.

10. You must build a model of electricity demand for a campus from: outdoor temperature, humidity, a temperature–humidity index (computed from the first two), hour of day, day of week, number of students on campus, and total floor area of buildings in use. Two of these columns have 15% missing values. The model will be used both to forecast next week's demand *and* to argue in a committee that switching off one building would save a stated amount. Design the analysis. For each decision cite a measured number from these notes.

12 Solutions

1. Expand $S(b) = y^\top y - y^\top Xb - b^\top X^\top y + b^\top X^\top Xb$. The two middle terms are 1×1 and transposes of each other, hence equal, so $S(b) = y^\top y - 2b^\top X^\top y + b^\top X^\top Xb$.

Rule one: $b^\top c = \sum_j b_j c_j$, so $\partial / \partial b_k = c_k$ and the gradient is c. Rule two: for symmetric A, $b^\top Ab = \sum_j \sum_k b_j A_{jk} b_k$; the terms involving b_m are $A_{mm} b_m^2$ and $2A_{mk} b_m b_k$ for $k \neq m$, so $\partial / \partial b_m = 2 \sum_k A_{mk} b_k$ and the gradient is $2Ab$. $X^\top X$ is symmetric because $(X^\top X)^\top = X^\top X$.

Hence $\nabla S = -2X^\top y + 2X^\top Xb$, and $\nabla S = 0$ gives $X^\top X \hat{\beta} = X^\top y$.

Minimum: $\nabla^2 S = 2X^\top X$ and $v^\top X^\top Xv = \|Xv\|^2 \geq 0$, so the Hessian is positive semi-definite everywhere and S is convex; every stationary point is a global minimum.

Uniqueness: the solution is unique iff $X^\top X$ is invertible, which by the argument of Section 3.6 holds iff $\text{null}(X) = \{0\}$, that is iff X has full column rank $p + 1$. If $\|Xv\| = 0$ for some $v \neq 0$, then $\hat{\beta}$ and $\hat{\beta} + v$ give identical fitted values and identical SSE.

2. (a) $\sum x_1 = 10$, $\sum x_2 = 10$, $\sum x_1^2 = 30$, $\sum x_2^2 = 30$, $\sum x_1 x_2 = 1 + 6 + 6 + 16 = 29$, $\sum y = 54$, $\sum x_1 y = 5 + 30 + 42 + 80 = 157$, $\sum x_2 y = 5 + 45 + 28 + 80 = 158$. So

$$X^\top X = \begin{pmatrix} 4 & 10 & 10 \\ 10 & 30 & 29 \\ 10 & 29 & 30 \end{pmatrix}, \qquad X^\top y = \begin{pmatrix} 54 \\ 157 \\ 158 \end{pmatrix}.$$

(b) Cofactors: $A_{11} = 30 \cdot 30 - 29^2 = 59$, $A_{12} = -(10 \cdot 30 - 29 \cdot 10) = -10$, $A_{13} = 10 \cdot 29 - 30 \cdot 10 = -10$, so $\det = 4(59) + 10(-10) + 10(-10) = 236 - 200 = 36$. Also $A_{22} = 4 \cdot 30 - 100 = 20$, $A_{23} = -(4 \cdot 29 - 10 \cdot 10) = -16$, $A_{33} = 4 \cdot 30 - 100 = 20$. Hence

$$(X^\top X)^{-1} = \frac{1}{36} \begin{pmatrix} 59 & -10 & -10 \\ -10 & 20 & -16 \\ -10 & -16 & 20 \end{pmatrix}.$$

(c) $36 b_0 = 59(54) - 10(157) - 10(158) = 3186 - 1570 - 1580 = 36$; $36 b_1 = -10(54) + 20(157) - 16(158) = -540 + 3140 - 2528 = 72$; $36 b_2 = -10(54) - 16(157) + 20(158) = -540 - 2512 + 3160 = 108$. So $\hat{\beta} = (1, 2, 3)$ and $\hat{y} = 1 + 2x_1 + 3x_2$.

(d) Fitted values $(6, 14, 13, 21)$, residuals $e = (-1, +1, +1, -1)$. Then $\sum e_i = 0$, $\sum x_{1i} e_i = -1 + 2 + 3 - 4 = 0$ and $\sum x_{2i} e_i = -1 + 3 + 2 - 4 = 0$. All three zero, so $\hat{\beta}$ satisfies the normal equations.

(e) SSE $= 4$. $\bar{y} = 13.5$, so SST $= 8.5^2 + 1.5^2 + 0.5^2 + 6.5^2 = 117$. Hence $R^2 = 1 - 4/117 = 0.9658$ and, with $n - p - 1 = 1$, $R^2_{\text{adj}} = 1 - (4/1)/(117/3) = 1 - 4/39 = 0.8974$. The gap between the two is large because three parameters on four rows leave one degree of freedom.

```
X'X =
 [[ 4 10 10]
 [10 30 29]
 [10 29 30]]
X'y = [ 54 157 158]      det(X'X) = 36
36 * inv(X'X) =
 [[ 59. -10. -10.]
 [-10.  20. -16.]
 [-10. -16.  20.]]
beta = [1. 2. 3.]      fitted [ 6. 14. 13. 21.]
residuals [-1.  1.  1. -1.]      X'e = [0. 0. 0.]
SSE 4.0000  SST 117.0000  R2 0.965812  adj R2 0.897436
LinearRegression agrees to 1.78e-15
```

3. $\bar{x}_1 = \bar{x}_2 = 2.5$, $\bar{y} = 13.5$. Centred: $x_1^c = (-1.5, -0.5, 0.5, 1.5)$, $x_2^c = (-1.5, 0.5, -0.5, 1.5)$, $y^c = (-8.5, 1.5, 0.5, 6.5)$. Then $S_{11} = S_{22} = 2.25 + 0.25 + 0.25 + 2.25 = 5$, $S_{12} = 2.25 - 0.25 -$

$0.25+2.25=4$, $S_{1y}=12.75-0.75+0.25+9.75=22$, and $S_{2y}=12.75+0.75-0.25+9.75=23$. The determinant is $25-16=9$, so

$$b_1=\frac{5(22)-4(23)}{9}=\frac{18}{9}=2, \qquad b_2=\frac{-4(22)+5(23)}{9}=\frac{27}{9}=3,$$

and $b_0=13.5-2(2.5)-3(2.5)=13.5-12.5=1$. The same answer with far less arithmetic.

For $p=3$ the centred route inverts a 3×3 instead of a 4×4, and a 4×4 determinant alone is four 3×3 determinants. Always centre. The only cost is one extra line for the intercept.

```
centred cross-product matrix [[5.0, 4.0], [4.0, 5.0]]   det 8.999999999999998
centred X'y [22. 23.]
slopes [2. 3.]   intercept 1.0
```

4. (a) Invertible, $\text{rank}(X)=4$ (three predictors plus the intercept).

(b) Not invertible. Height in inches is $0.3937\times$ height in cm exactly, so $\text{rank}(X)=3$ against the 4 required. Fix: drop one of the two. `LinearRegression` will not raise; it will split the height effect between the two columns in whatever way `lstsq` finds smallest in norm, and both coefficients will be meaningless.

(c) Not invertible. The four dummy columns sum to 1 in every row, which is the intercept column, so $\text{rank}(X)=4$ against the 5 required. Fix: `OneHotEncoder(drop='first')`, or drop the intercept. Again no error is raised, and no coefficient, standard error or p-value from that fit means anything.

(d) Not invertible: $\text{rank}(X)\le n=12<31$. The null space has dimension at least 19, so there is a 19-dimensional family of coefficient vectors that all fit the twelve rows exactly. Fix: reduce p (selection), or regularise (ridge, lasso). `LinearRegression` returns the minimum-norm member of that family and a training R^2 of 1.

5. (a) Equation (15) says $b_1^{\text{simple}}=b_1+b_2\delta=-1+4(0.4)=-1+1.6=+0.6$. Both reported numbers are correct and consistent.

(b) The -1. The advertisement describes a conversion that adds a bedroom *without changing the floor area*, which is precisely the intervention the partial coefficient describes. The $+0.6$ answers a different question: "if I compare a randomly chosen four-bedroom flat with a randomly chosen three-bedroom flat, how much more does the first cost?" — and that comparison silently includes the extra $20\,\text{m}^2$ that four-bedroom flats tend to have. Using $+0.6$ to justify the conversion is the omitted-variable error, and it has the wrong sign.

6. (a) $\beta^*_{\text{income}}=0.42\times 30000/18000=0.70$; $\beta^*_{\text{size}}=3200\times 1.8/18000=0.32$.

(b) Income, by a factor of about 2.2. The precise meaning: a household one standard deviation (30,000 in rupees) above average in income spends 0.70 standard deviations (12,600 in rupees) more, *holding household size fixed*; a household one standard deviation (1.8 persons) larger spends 0.32 standard deviations (5,760 in rupees) more, holding income fixed. It does not mean income causes spending, and it does not mean income would still rank first in a model with different predictors.

(c) b becomes 420 (a thousand times larger, since a unit of x is now a thousand times bigger), s_j becomes 30 (a thousand times smaller), and $\beta^*=420\times 30/18000=0.70$, unchanged. That invariance is the whole point of the statistic, and Section 6 checks it numerically at 10^4 on the `bmi` column.

7. (a) $\text{VIF}_3=1/(1-0.96)=25$; the standard error is inflated by $\sqrt{25}=5$.

(b) Standard errors fall like $1/\sqrt{n}$, so you need 25 times as many observations to undo a variance inflation of 25.

(c) Two sentences, honestly written: *"The coefficient of x_3 is not identified by these data: its variance inflation factor is* 25 *and its* 95% *bootstrap interval,* $[-0.4, +2.1]$*, includes zero, so we cannot establish even its sign. The model's predictive accuracy is unaffected — cross-validated* R^2 *is* 0.71 *— and we report it as a predictive model only."* The measured precedent is in Section 7.3: two fits differing by 2.55 in a coefficient scored $R^2 = 0.989$ each.

8. (a) $500 \times 0.92^{12} = 500 \times 0.3677 = 184$ complete rows.

(b) 184 rows for 13 parameters is workable but you have thrown away 63% of the data for no reason, and every standard error is inflated by roughly $\sqrt{500/184} = 1.65$. On the measured diabetes example, deletion at 10% cost 0.0698 of test R^2 and at 25% it produced -2.15.

(c) MAR if "looked healthy" is captured by other recorded columns (age, blood pressure, symptoms), because then the missingness depends only on observed data and a model conditioning on those columns can undo the bias. MNAR if the clinicians were responding to something they saw and did not record. Under MAR, `IterativeImputer` is appropriate. Under MNAR, no imputation is safe and the honest move is a sensitivity analysis plus a prominent caveat.

(d)

```
pipe = Pipeline([
    ("imp", IterativeImputer(random_state=0, add_indicator=True)),
    ("scale", StandardScaler()),
    ("model", LinearRegression()),
])
cross_val_score(pipe, X, y, cv=KFold(5, shuffle=True, random_state=0))
```

Every step is inside because every step learns numbers from data: the imputer learns per-column regressions, the scaler learns means and standard deviations. The indicator columns are included because the missingness here is *not* MCAR — "no test ordered" is itself clinical information. The measured leak from getting this wrong is small (+0.0002 to +0.0042), but it is free to avoid.

9. Four errors, in increasing order of seriousness.

"R^2 was 0.62*, up from* 0.55*, so the larger model is better."* R^2 cannot decrease when you add predictors — Equation (18) — so this comparison carries no information at all. Measured: adding twenty columns of pure noise to 80 rows took R^2 from 0.5911 to 0.6935 while held-out R^2 fell from +0.2488 to −0.0794. *Demand*: adjusted R^2 or, better, cross-validated R^2 for both models.

"Coefficient -1.09*, so lowering cholesterol increases readmission risk."* The coefficient is conditional on the other nine variables, several of which are components or ratios of cholesterol itself. Section 7 measured VIF = 59.2 for exactly this column, a bootstrap interval of $[-2.10, +0.06]$ that includes zero, and a sign that flipped in 3.1% of resamples. *Demand*: VIFs, a bootstrap interval, and the added-variable plot. Then delete the sentence.

"LDL has the largest coefficient at $+22.7$*, so it is the strongest predictor."* Raw coefficients are in units of y per unit of x_j and are not comparable. *Demand*: $\beta^* = b_j s_j / s_y$. On the diabetes table this reordering moved one predictor from sixth to first.

"Residuals had mean zero and were uncorrelated with the fitted values, confirming the model is well specified." Both properties are forced by the normal equations (Section 3.9) and hold for a straight line fitted to a parabola, which we measured at $R^2 = 0.0000$ with a residual mean of -2.66×10^{-16}. *Demand*: the residual plot itself, a Q–Q plot, and a scale–location plot.

A rewrite: *"We fitted a linear model of readmission on ten clinical variables. Cross-validated* R^2 *was* X *against* Y *for the five-variable model; adjusted* R^2 *was* A *against* B*. Four of the ten predictors have variance inflation factors above* 10*, and their coefficients are not individually identified — the bootstrap interval for total cholesterol,* $[-2.10, +0.06]$*, includes zero — so we*

report this as a predictive model and draw no conclusions from individual coefficients. Standardized coefficients, which are comparable across predictors, rank them as follows. Residual diagnostics (Figure n) show no curvature but mild heteroscedasticity, so prediction intervals at the high end should be treated as optimistic."

10. A defensible design, with the evidence.

First, split the purpose in two, because it is two models. Forecasting next week's demand and arguing that a building may be switched off are different jobs, and Section 7.3 is the reason: two fits whose coefficients differed by 2.55 scored $R^2 = 0.989$ each. Say so explicitly in the committee paper.

The temperature–humidity index. Drop it, or drop temperature and humidity. It is computed from the other two, so if the relation is exact the design matrix is rank deficient (Section 3.7: rank 3 of 4, condition number 7.7×10^{18}, coefficients not unique) and if it is nearly exact the VIFs will be enormous. Print the VIFs and check.

Hour of day and day of week. Nominal, not numeric — hour 23 is adjacent to hour 0. One-hot encode, with `drop='first'` if anyone will read the coefficients, and consider sine and cosine of the hour if a smooth daily cycle is expected.

Students on campus and floor area in use. These will be strongly correlated with each other and with hour of day. Compute VIFs before interpreting anything. If the committee question is specifically "what does this one building contribute", that is a coefficient question, so the collinearity must be dealt with, not ignored — combine, drop, or accept a wide interval and report it.

The two columns with 15% *missing.* Complete-case analysis would keep $0.85^2 = 72\%$ of rows if only those two columns are affected — survivable — but there is no reason to pay it. Use `IterativeImputer` inside the pipeline; measured on ten columns at 25% it recovered +0.4038 against a complete-data +0.3929. Do not fill with zero: measured +0.061.

Validation. Cross-validate with a *time-aware* split, because electricity demand is a time series and shuffled folds would let the model see next Tuesday while predicting last Tuesday. Report the fold-to-fold spread: on the diabetes pipeline it ran from 0.28 to 0.49, and a single split would have reported any of those with equal confidence. Then look at the residual plot against time and against temperature, because a straight line fitted to a curve has $R^2 = 0.0000$ and perfect summary statistics.

For the committee. Report the standardized coefficients with bootstrap intervals, state which ones straddle zero, and give the predicted saving as an interval rather than a number.

13 References and further reading

All links were checked on 22 August 2026. Free resources are marked [**free**].

Textbooks

- James, Witten, Hastie, Tibshirani and Taylor, *An Introduction to Statistical Learning with Python.* [**free**] **The main reference for this lecture.** §3.2 multiple linear regression — §3.2.1 estimating the coefficients and the "holding the others fixed" discussion, §3.2.2 the four important questions including the F test and R^2; §3.3 other considerations, of which §3.3.3 covers non-linearity, outliers, high leverage and **collinearity with variance inflation factors**; §3.4 the marketing worked example; §6.1.3 on adjusted R^2 and why training error cannot select a model. `https://www.statlearning.com/`
- Deisenroth, Faisal and Ong, *Mathematics for Machine Learning.* [**free**] Ch. 9 "Linear Regression": §9.1 the problem formulation, **§9.2 parameter estimation** — the matrix derivation

of this lecture done in full, including the maximum-likelihood route and the design-matrix condition; §9.2.4 on the connection to MLE. §5.5 gives the vector-calculus identities used in Section 3. `https://mml-book.github.io/`

- Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006. §3.1 linear basis function models; §3.1.1 derives the normal equations and introduces the Moore–Penrose pseudo-inverse, which is what `lstsq` returns when $X^\top X$ is singular (Sections 3.7 and 3.8 here).
- Montgomery, Peck and Vining, *Introduction to Linear Regression Analysis*, Wiley. The standard reference in the vocabulary the examination uses: Ch. 3 multiple regression in matrix form, including **standardized regression coefficients**; Ch. 4 model adequacy checking, including **partial regression (added-variable) plots**; Ch. 9 multicollinearity, including the derivation of the variance inflation factor.
- Hastie, Tibshirani and Friedman, *The Elements of Statistical Learning*, 2nd edition, Springer. §3.2 linear regression models and least squares; §3.2.3 "regression by successive orthogonalisation", which is the Frisch–Waugh–Lovell construction of Section 5.4; §3.4 on ridge and lasso as the answer to Section 7.
- Müller and Guido, *Introduction to Machine Learning with Python*, O'Reilly, 2016. §2.3.3 linear models, including `LinearRegression` on the diabetes-style tables used here and the transition to ridge and lasso. Notebooks [**free**]: `https://github.com/amueller/introduction_to_ml_with_python`
- Stef van Buuren, *Flexible Imputation of Missing Data*, 2nd edition. [**free**] The reference for Section 8: §1.1–1.3 on listwise deletion, mean imputation and the indicator method, with a clear account of exactly what each one biases; §2.2.4 on MCAR, MAR and MNAR; Ch. 4 on the MICE algorithm that `IterativeImputer` implements. `https://stefvanbuuren.name/fimd/`

Online courses

- **NPTEL, Linear Regression Analysis** — Prof. Shalabh, IIT Kanpur. The closest match anywhere to this lecture, in exactly the vocabulary the examination uses: the multiple regression model in matrix form, the normal equations, standardized coefficients, multicollinearity and diagnostics. [**free**] `https://nptel.ac.in/courses/111104074`
- **NPTEL, Introduction to Machine Learning** — Prof. Balaraman Ravindran, IIT Madras; the reference course for the semester, with a linear-regression week. [**free**] `https://nptel.ac.in/courses/106106139`
- **SWAYAM** — the national portal hosting NPTEL. [**free**] `https://swayam.gov.in/`
- **Google Machine Learning Crash Course** — the "Linear regression" module, short and practical. [**free**] `https://developers.google.com/machine-learning/crash-course`

Videos

- StatQuest with Josh Starmer, *Multiple Regression, Clearly Explained!!!* — the extension from one predictor to many at a whiteboard, with the R^2 and F comparison between nested models. Watch this first. [**free**] `https://youtu.be/EkAQAi3a4js`
- StatQuest with Josh Starmer, *Linear Regression, Clearly Explained!!!* — the one-predictor case, if you want the ground under Section 3 re-laid before you start. [**free**] `https://youtu.be/nk2CQITm_eo`
- CampusX, *Multiple Linear Regression | Part 2 | Mathematical Formulation From Scratch* — the derivation of $\hat{\beta} = (X^\top X)^{-1} X^\top y$ worked through slowly, with the code that implements it. [**free**] `https://youtu.be/NU37mF5q8VE`

- Ben Lambert, *Variance Inflation Factors: testing for multicollinearity* — eight minutes on Equation (17) and how to read the result. [**free**] `https://youtu.be/0SBIXgPVex8`

Documentation and interactive explainers

- Victor Powell and Lewis Lehe, *Ordinary Least Squares Regression explained visually* — drag the points and watch the fitted plane and the residuals move. The best five minutes you can spend on Section 3. [**free**] `https://setosa.io/ev/ordinary-least-squares-regression/`
- Jared Wilber, *Linear Regression* (MLU-Explain) — an interactive account of least squares, R^2 and the multivariate case. [**free**] `https://mlu-explain.github.io/linear-regression/`
- Penn State, *STAT 462* §10.7, "Detecting Multicollinearity Using Variance Inflation Factors" — a full worked blood-pressure example with the VIFs computed by hand and two predictors removed. [**free**] `https://online.stat.psu.edu/stat462/node/180/`
- scikit-learn user guide: *Linear Models* (§1.1, with `LinearRegression` and the note that it uses `lstsq`); *Imputation of missing values* (§8.4, `SimpleImputer`, `IterativeImputer`, `add_indicator`); *Common pitfalls and recommended practices.* [**free**] `https://scikit-learn.org/stable/modules/linear_model.html` `https://scikit-learn.org/stable/modules/impute.html` `https://scikit-learn.org/stable/common_pitfalls.html`
- statsmodels, *Linear Regression* — if you want the classical output table (standard errors, t, p, confidence intervals, F-statistic, condition number) that scikit-learn deliberately does not print. [**free**] `https://www.statsmodels.org/stable/regression.html`

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.;Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.;Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.;Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.